Computer Standards & Interfaces 84 (2023) 103697

Contents lists available at ScienceDirect

Computer Standards & Interfaces

journal homepage: www.elsevier.com/locate/csi

Microservice based scalable IoT architecture for device interoperability

Chouhan Kumar Rath [a,*], Amit Kr. Mandal [b], Anirban Sarkar [a]

[a] Department of Computer Science and Engineering, National Institute of Technology, Durgapur, 713209, West Bengal, India
[b] Department of Computer Science and Engineering, SRM University, Amaravati, 522240, Andhra Pradesh, India

ARTICLE INFO

Keywords:
IoT architecture
IoT template
Microservices
Scalability
Interoperability
Event-B

ABSTRACT

The Internet of Things (IoT) revolutionizes the technology landscape by enabling a wide spectrum of services and applications, characterized by a large number of devices, communication protocols, and data formats. Seamless integration among various IoT-enabled technologies is the most challenging task as the technical standards are disjoint. This often results in monolithic structures with very poor scalability. Further, data heterogeneity in IoT networks increases the measure of multidimensionality, which poses a critical challenge of sharing data with other business applications. Therefore, IoT-based solution requires an architectural framework supported by a large number of independent and specialized microservices towards providing sufficient scalability and interoperability. In this manuscript, a layered architectural framework is proposed where a novel device template concept is introduced to enable flexible and interoperable integration of microservices with IoT devices. The proposed model is formally validated using Event-B in the Rodin platform with the help of proof obligations. Again, a service aggregation algorithm is proposed to reduce the latency and increase the reuse of microservices. Further, the proposed architectural framework is implemented and compared with other similar frameworks. The experimental result shows that the proposed architectural framework enables enhanced interoperability, scalability, and reusability of IoT devices and microservices.

1. Introduction

In general, the Internet of Things (IoT) architectures encompass a complex network of physical devices and cloud-based services for storing and analyzing a large volume of data produced by the devices. A cloud service-based IoT architecture defines how the IoT application should interact with the outside world by facilitating efficient data sharing using autonomous web services. However, with the increasing popularity of API economy [1] the classic SOA-based IoT solutions lose their granularity as it becomes too complicated with a large number of interfaces [2,3]. To address this, researchers in industry and academia devising suitable IoT architecture to design, develop and deploy End-to-End IoT applications. But, the rapidly evolving domain demands seamless interaction among various types of devices, communication protocols, and services [4]. In general, a typical IoT system consists of various types of sensors and actuators. Thus, the data format and communication protocols are also different, and this heterogeneity in IoT environment increases the measure of complexity, which poses a critical challenge on the aggregation of data and services with other business applications [5]. Further, the constituent service after the aggregation should be accessible with minimum response time, and it should be easily upgradable. Again, services in an architectural framework should be reusable as it helps in achieving lower creation and updating time, which also enables IoT systems to become easily integrable with newer technologies [6]. In addition to this, it should support the flexibility of migrating from one computing environment to another [7]. Finally, to ensure pertinent, secure, and efficient functionalities of IoT applications, continuous upgradation, integration, and maintenance is necessary [8]. These necessitates the need for an architectural framework for creating IoT applications that can be developed, managed, and decomposed into specialized services. For this purpose, microservices are utilized, which comprises data-driven interfaces and runs as an independent process which is self-sufficient [9].

Microservice-based IoT architecture can enable creating a set of atomic services for some specific IoT functionality. These functionality specific services can be of various types such as: a set of services could be devised to collect sensor and controller details and present it in the form of data instead of devices [5], whereas another set of microservice can process the IoT sensor data by applying the constraints defined on that data [10]. Therefore, granular control is attainable for individual IoT services as the corresponding microservices are independent of each other. This independence among IoT services can enable de-centralize data management, flexible development, and easy update [8]. Therefore, convergence of microservice and IoT brings numerous advantages,

* Corresponding author.
E-mail addresses: chouhanrath.nitdgp@gmail.com (C.K. Rath), amitmandal.nitdgp@gmail.com (A.K. Mandal), sarkar.anirban@gmail.com (A. Sarkar).

https://doi.org/10.1016/j.csi.2022.103697
Received 24 February 2022; Received in revised form 28 June 2022; Accepted 12 October 2022
Available online 21 October 2022
0920-5489/© 2022 Elsevier B.V. All rights reserved.

but its implementation using an architectural framework is far more challenging. To strengthen the adaption of microservices in IoT, the architectural framework should enable efficient scaling, unlike monolithic applications upscaling a microservice based IoT system may involve scaling all accompanying components and services [11,12], it indicates that all the associated services need to be scaled at once. It can be avoided by identifying and scaling appropriate microservices. To facilitate this, the architectural framework should support identifying specific services for scaling from a large pool of services, and it should also ensure that the scaled service meets all the hardware constraints of IoT devices. In an IoT environment, devices are primarily considered as resources [13,14]. Thus, prioritizing the resources is another challenging task, as the devices are heterogeneous, de-centralized, and autonomous entities. Further, in IoT based microservices environment, a rough dependency may destroy the scalability [2]. To avoid this, the architectural framework should support mapping service requests to the microservice topologies. Again, in "silos" or closed IoT ecosystems, it is difficult to integrate new IoT devices and services, thus cannot be scaled up as and when required [15]. To address this issue, the IoT architectural framework should allow effective data exchange across different and overlapping devices and systems. This is a very challenging task as with the increasing number of devices, the IoT network becomes more complex, particularly with the interoperability needed to be defined at the different levels [16]. Microservice development is often organized on business capabilities rather than technical capabilities. To maintain autonomous development, each new demand should be addressed by only one microservice [1]. Requirements that involve more than one microservices indicate an ineffective design and need for architectural changes. Further, the consumer of the services may require fine-grained or coarse-grained services. For this purpose, the architectural framework must enable an efficient service aggregation mechanism, where API gateways can be used as a single-entry point for the combined services. This will route all the requests to the appropriate microservices, whether it is fine-grained or coarse-grained [17].

To address these challenges, a microservice-enabled IoT architecture has been proposed in this paper. It comprises four layers: *Device Layer, Template Layer, Microservice Layer* and *Application Layer*. The *Device Layer* consists of IoT devices that are responsible for monitoring and interacting with the physical world. In this layer, a profile for each device can be created using its features and capabilities. Profiling such devices help in enhancing the interaction capabilities and conceptualizing the relationship among them. In *Template Layer*, a novel template-based concept for IoT devices is introduced. A digital profile is created through the template, which defines the functionalities, characteristics, and features of a type of device used in IoT based system. Generally, vendors are following different configuration profiles for the devices. There are no standard templates to describe generic features of IoT devices. Sometimes, the devices also have significant differences among them in terms of capabilities and the communication technologies they use. It signifies the objective of this paper to make use of a universal template that improves the integration of devices and data-level interoperability. The *Microservice Layer* provides the necessary support of structuring the IoT application by enabling the creation and aggregation of microservices to provide on-demand services. In this layer, the applications are split into small independent units, and separate microservices are created for every single unit. A service aggregation mechanism is presented in this layer, which enables the development of scalable services from a collection of tiny microservices. This pattern also supports the reusability of both device and microservices, this in turn, facilitates faster development and deployment of complex IoT applications. Whereas the user-specific services are delivered by the *Application Layer* with the support of client interaction interfaces. The syntax and semantics of the proposed architectural framework are formally conceptualized using Event-B modeling language, and the crucial concepts such as: device profile, device templates, microservices, and data streams are validated using the proof obligations (POs) in the Rodin platform. To demonstrate the effectiveness of the proposed architectural model, it is implemented to simulate with a case study based on the Smart City application. Further, the proposed model is compared with two other similar approaches. The obtained result shows that the proposed architectural framework significantly reduces the latency after aggregation, supports efficient scaling and reuse of services. Further, it is also shown that the device templates used in the architecture not only enhance the integration and interoperability but it also enables the easy update of devices and corresponding microservices.

2. Related work

With the increasing adaption of IoT-based applications, various types of devices are used, which often result in a highly heterogeneous ecosystem. This heterogeneity can be observed at every stage, from device configuration to data collection and data processing. For this purpose, it requires harnessing all the resources into a single framework to formulate a reliable, scalable, interoperable, and cost-effective solution. In this direction, several researchers proposed IoT architectures based on IoT middlewares and SOA [4,5,18] which are suitable for many IoT applications such as Smart Home, Industrial Automation, Connected Healthcare, Smart Farming, etc. Y. Mesmoudi et al. [18] presented a SOA-based middleware that provides a generic data format for searching the connected devices through various network interfaces which is available as a service. Similarly, S. Pulparambil et al. [3] used SOA in their approach for complex interactions and computations of IoT services with the support of core functionalities such as discovery, composition, and access. By leveraging the modular characteristics of edge computing, A. Javed et al. [5] also provided an open standard IoT framework for service/device discovery, where configuration, and data management is achieved by utilizing the flexibility of the edge computing. A majority of these approaches are monolithic architectures as the modules are independent, uniform, and interconnected in a single unit [15]. However, it is not applicable in a large-scale complex IoT system because of the difficulty in upgradation, maintenance, and continuous integration. Therefore SOA based applications for IoT gained popularity due to their inherent support of interoperability and compositability [4,18]. But many IoT service providers presently use microservices as the core component of IoT [7,12]. It allows the system to be scalable for the specified services. It also helps in attaining required agility for continuous deployment, and fault tolerance to meet the future IoT middleware demands [8].

The majority of the microservice based IoT architectures studied in literature are expressed in natural language rather than following the standard formal specification [19]. This may lead to ambiguity and incorrect implementation, limiting the benefits of architectural patterns and making it difficult to create correct microservice applications. Due to ambiguous explanations, developers and service providers may find it challenging to adopt such a system and understand what an existing implementation offers. Hence, formal methods for modeling and validating such requirements are desirable. A. Le et al. [20] uses Fuzzy If-Then rules and B-notations for representation of ambiguous requirements. First, they present the requirements in set theory. Then they developed system refinement, which includes a set of transitioning rules from Fuzzy If-Then rules to B-notations. M. Camilli et al. [21] presents a formal semantics based on PertriNet for describing and testing microservices-based process flows. To examine the features of microservice-oriented applications, the authors used Time Basic Petri nets, a special version of Petri nets that supports the definition of temporal constraints. Similarly, R. Petrasch [22] presented a formal model for the microservices architecture using UML profiles. M. Singh et al. [23] presents a formal analysis of traffic management using Z-notation, and their approach is intended to eliminate the ambiguity using mathematical theorems and set theory. However, these formal

methodologies are primarily used for representation rather than validation. For this purpose, many researchers used Event-B modeling language for the representation and validation of the IoT system. Event-B is based on the concept of events or transitions, and its primary goal is to assist in the creation of correct-by-construction systems. C. Rath et al. [24] presents IoT resources and the device description model, where the Event-B is used for the correctness of the model. M. Diwan et al. [25] proposed an Event-B based approach for modeling and validating IoT communication protocols. Further, the verified model is used in different IoT protocols such as MQTT, MQTT-SN, and CoAP. I. Sadoune et al. [26] presents an Event-B semantics for formalizing the composition of web services. The Event-B technique is used to create a transactional Business Process Execution Language (BPEL) process that addresses the scope and limitations of the BPEL language specification. I. Tounsi et al. [27] presented a formal approach for modeling and transforming message-oriented SOA architecture patterns into formal specifications. They have used Event-B formalization which allows characterizing both structural and behavioral features of SOA at a high level of abstraction in order to give the correctness of the model. Through the Event-B method, J. Bryans et al. [28] provided a formal analysis of business process models. They took into account both control and data flow models and demonstrated how the properties are verified by automatic proofs. The Event-B based formalization method are mostly used for developing formal specifications in a variety of contexts, including web service composition, SOA design patterns, BPMN models, etc. However, formal specifications for microservice based IoT applications are still unexplored. Therefore, this paper presents a formal model for the microservice based architectural framework in order to prove the correctness of the model.

IoT ecosystem connects a large number of sensors, actuators, and other devices, posing the design and growth issues of the system to meet scalability, interoperability, and reusability. The main objective of making the device/service scalable is to fulfill the ever-changing demands since the user's interest always changes with time and environmental conditions. D.Correl-Plaza et al. [29] provides an approach for processing and analyzing data from heterogeneous resources by combining real-time stream processing and information processing. Antonio. J. Fernández et al. [30] presents a microservice architecture and deployed in data acquisition system in order to acquire data from various applications whose user interfaces run distributed across multiple devices. In S. Trilles et al. [11] a centralized management platform is devised which supports a wide range of devices, data, and applications. They have used microservices and serverless paradigm, which ensure a scalable and stable platform. C. Lai et al. [7] and A. Javed et al. [5] presents a scalable microservice architecture for a smart city application that can handle the growing number of IoT devices, users, and computing environment. The "InterSCity", an open-source platform based on microservice architecture, is suggested for allowing rapid development of various services, systems, and applications required for the smart IoT. The scalability can be achieved in this platform due to the modular design and technology, independent deployment, and functional decomposition [12]. Many context-aware IoT applications are also examined in a smart environment to determine their capabilities in terms of real-time decision-making and scalability [31]. These experiments are being carried out to demonstrate scalability and dynamic load adaptability for IoT applications. To create highly scalable Fog-based IoT systems, an oneM2M platform was built by leveraging the container technology. In the Fog-nodes certain number of oneM2M middle node containers are deployed. The traffic is subsequently distributed across all serving instances by the load balancer. A scaling server is utilized, which monitors the condition of each Fog-node based on its CPU and memory use, further, it scales in and out automatically [32]. Similarly, a three layered architecture is proposed in paper [33] using Network Function Virtualization (NFV). It provides virtual machines towards scaling the IoT applications using physical devices. R. F. Maia et al. [34] have developed a crop management IoT platform that combines native cloud services and Microservices. In the field of smart farming, this platform makes use of microservices and serverless or container-based virtualization. C. Santana et al. [6] presented a reactive microservices architecture for designing IoT applications. It contains a software framework for meeting IoT application availability requirements at runtime. For smart agricultural applications, the platforms serve as an intermediary layer. H. Chegini et al. [35] proposed a microservice based IoT framework for creating a context-aware intelligent decision-making systems. G. Ortiz et al. [36] proposed a layered wise context aware IoT architecture for real-time data processing by using agent oriented software, SOA, and complex event processing (CEP). N. Coulson et al. [37] designed a prototype auto-scaling system for microservice-based web applications that can learn from the previous service experience. They have developed a pipeline for microservice auto-scaling and a supervised learning model for a recommendation of auto-scaling actions. The authors have emphasized the benefits of microservices adoption, such as robustness and scalability, however there is no evidence of how these benefits are realized. The above discussed architectures provide various design principles for achieving scalability in various components of IoT. Most of them are dealing with the applications are running on a single server and load balancing in a few servers. But, there may have components of an IoT application deployed in different cloud platform, loaded on different hardware, and running in different virtualization platforms. All the components have to be scaled or able to identify the individual components needed to scale in large scale IoT systems. Further, a scalable IoT system should also support the exchange of data between overlapping devices and systems.

In this context, interoperable architectures enable communication between heterogeneous IoT devices and systems, allowing numerous devices, systems, platforms, and networks to work together to achieve a common goal [14]. M. Noura et al. [16] summarizes the issues and facilities adding interoperability in different IoT platforms. S. K. Roy et al. [14] presents an approach for seamless integration of peripherals with heterogeneous devices towards achieving interoperability in IoT. Another interoperable and decentralized IoT architecture is being designed to meet the heterogeneity of IoT device requirements and to enable seamless integration of new devices into existing IoT applications [38]. T. Vresk et al. [39] proposes a microservice-based architecture with the aim of preserving cohesiveness across multiple devices. A. Benayache et al. [40] proposed a microservice based middleware architecture using Artificial Neural Network (ANN) that addresses the heterogeneity and interoperability issues of services. G. Bellot et al. [41] shows how a microservice architecture deals with heterogeneous data. It describes microservices by Web of Things (WoT) standards which facilitate interoperability with other microservices. N. Gavrilovic et al. [42] analyze different types of IoT architecture like cloud-based, service-oriented, and layered architectural applications in various areas of IoT. N. Dipsis et al. [43] proposed a RESTful middleware (eVATAR+) for AI controlled smart devices which adapts intelligently to changes of IoT environment. Similarly, G. Porruvecchio et al. [44] presents a two layered middleware, where the top level provides vertical features to simplify the development of complex IoT applications and services, and the lower level, a microservice architecture that handles all cross-cutting concerns, allowing developers to focus on domain functionalities. The increasing number of IoT platforms are providing new services, technological challenges are arising in order to provide better and faster service. Current initiatives have failed to achieve a high level of interoperability in the device, communication, and data processing layers since there are no standards for device setup and communication. All the design principles are emphasized to achieve interoperability but not concerned about scaling up while integrating a new device and service. They have added interoperability in a different perspective of IoT separately. However, there is a need for IoT framework which should support interoperability

among the devices, networks, services, and platforms for the exchange of information across different systems with a high level of efficacy.

Again in a scalable and interoperable IoT environment, managing resources is a challenging task. Resource management is necessary to make a large number of IoT devices easily discoverable and accessible in the IoT network. For device registration and status detection, W. Jin et al. [13] suggested an Open Connectivity Foundation (OCF)-based resource management method. The devices host the OCF resources to provide IoT services. The devices' information is published on the management platform via the registration policy, allowing users to identify resources and access the services. To overcome the challenges of limited computing capabilities at edge gateways R. Morabito et al. [45] devised a Lightweight Edge Gateway for the IoT (LEGIoT). The architecture is based on microservices, and the application of LEGIoT in real sensor network shows efficiency in resource management. M. Tavana et al. [46] discussed about implementation of Enterprise Resource Planning (ERP) in IoT systems that enable the IoT devices to manage, control, and process the data. M. Aslanpour et al. [47] presents rich information related to serverless computing on edge and summarizes the opportunities and challenges on resource consumption, and its management. On the edge computing platform, Li. Pan et al. [48] evaluated various resource management methodologies for microservice based IoT applications. They used Docker containers to construct microservices and conducted experiments on different scenarios to see how the microservice deployment strategies performed. Their findings show that as the service scenario changes, the computing resources of microservices in an edge computing network are varying drastically. D. Li et al. [49] proposed a fuzzy-based microservice computing resource scaling (FMCRS) method for managing the resource growth scale of microservices dynamically in order to satisfy their resource consumption needs in a timely manner. C. Liu et al. [50] presented an auto-scaling technique in which edge resource management is employed to determine the priority of microservices that need to be executed. The majority of recent research has concentrated on developing new microservice deployment mechanisms. However, there have been few discussions on offering efficient solutions for microservices service resource management in IoT. All the discussed architecture shows only the control and management of IoT edge resources whereas proper resource management may have dependencies over availability of resources and completion time of any activity [13]. Therefore, a system must enable to manage the resources by maintaining various dependencies in the dynamic environment [41]. This also enhances the reusability aspects of IoT objects and provides a suitable mechanism for sharing objects among various IoT applications [51]. In this direction M. Huang et al. [17] propose a service aggregation algorithm based on queuing delay utilization. The packets are divided into forwarding and aggregating packets, and the aggregating packets' service aggregation is completed by utilizing the forwarding packets' transmission time to ensure that the transmission volume and communication overhead are effectively reduced without adding additional latency. Similarly, P. Smet et al. [52] provides an on-demand service provision strategy that enables users to connect to a server, retrieve all necessary services from various locations, and deploy an instance of the requested service within a container.

Therefore, from the literature studied in this section it can be observe that an IoT architectural framework should allow flexibility in device integration, and further should combine with microservice architecture to enable continuous up-gradation, and easier maintenance. Due to the disparate nature of the communication protocols and data formats, the system should also support scalable and interoperable solutions for microservice-based IoT systems. The architectures that have been explored above presents different design principles to achieve scalability in different IoT components. Where, majority of them deal with applications that is hosted on a single server. However, components of an IoT application may be deployed on a separate cloud platform, loaded into a different piece of hardware, and running on a different platform. Hence, it is necessary to make a system where all the components are need to be scaled in large scale IoT applications. Again, majority of the studied techniques fall short of achieving a high level of interoperability in the device, communication, and data processing layers since it is difficult to attain a common consensus for device setup and communication mechanism. Further, majority of the design principles which emphasizes on achieving interoperability are difficult to scale-up. To address these issues the proposed model is made to support high levels of efficiency in information sharing across various platforms and high levels of interoperability among devices, networks, and services.

3. Proposed architectural model

The layered architecture for the IoT based on microservices is presented in this section. The architectural framework is divided into four different layers, which are: *Devices Layer, Template Layer, Microservices Layer,* and *Application Layer* as depicted in Fig. 1. These interdependent layers are helpful in conceptualizing different components of IoT and microservices. It also localizes the effects of changes to one part of the solution; thus, maintenance of application becomes easier, which in turn enhances the flexibility of the system. The detailed descriptions of the various layers are as follows:

3.1. Device Layer

This layer provides a semantic description of the features and capabilities of IoT devices. Profiling such devices enables maintaining efficient information exchange and relationships among them. The device profile includes various hardware parameters such as *device category, device Id, applicability, constraints, network interfaces*, etc. The semantic description of IoT devices helps to address the heterogeneity among various devices and networks. The various parameters of a device profile are:

(a) *Physical Properties:* IoT uses both static and mobile devices, and each one has its own set of deployments. For the recognition of IoT devices, it consist of an *identity, category* and *contextual information*. The devices are identified by its *name, unique id* and *category*.
(b) *Functional Characteristics:* The functional characteristics of IoT devices will provide information such as the relationships among the devices, their *applicability*, and *constraints*. The IoT devices offer broad functionalities with the collaboration of other devices. The *applicability* is decided based on which platform the device will work, and the *constraints* of IoT devices includes power, memory, network bandwidth, etc.
(c) *Observable Properties* It represents the data sensed by the physical entity or any event, its *Data type*, and *Dissemination* of data.
(d) *Network Interface Properties* It provides the information about the *Gateway address, Transmission protocols*, *Cloud information*, and the commands/methods through which the user can interact with the devices. There are many protocols used in different layers such as Infrastructure (6LowPAN, IPv4/IPv6), Transport (Wifi, Bluetooth, LPWAN), Discovery (Physical Web, DDS), Data Protocols (MQTT, CoAP, AMQP), and among others.

Profiling IoT devices means a base standard or a set of standard features associated with a particular device. After the device has been configured, the profile may be assigned to the user so that those features are available when the user logs in to that device. The above properties further enhance the ability of devices or applications to access, exchange, integrate, and utilization of data in a coordinated manner. Few properties that are visible to the user are borrowed from the device layer (as shown in Fig. 2) to make the device profile for IoT devices.

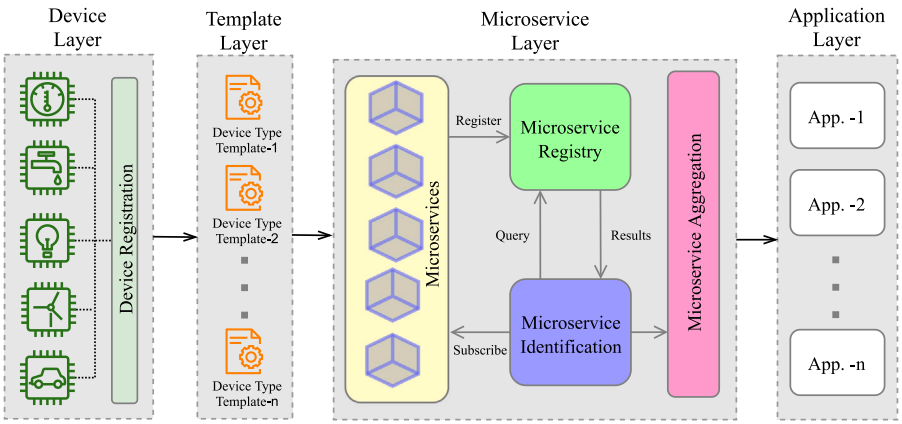


Fig. 1. The proposed layered architecture and its components for IoT as a whole.

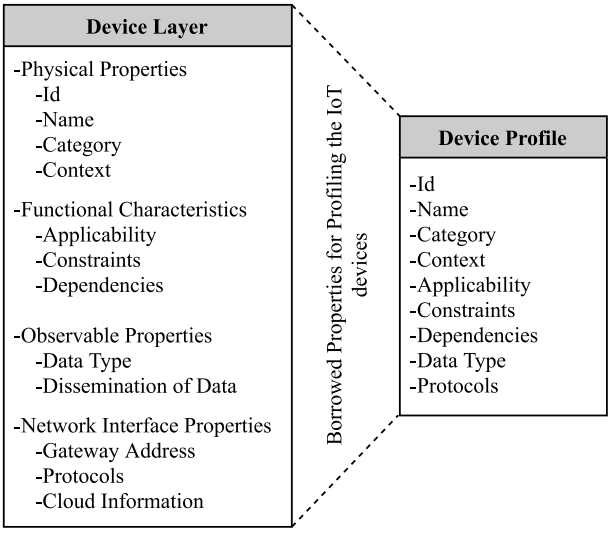


Fig. 2. Selected properties for Profiling IoT devices.

The IoT devices often support extensible plugins or a set of instructions to manage low-level device communication and provide APIs. Each device has a unique profile identity that allows it to be represented in the IoT network. At the time of initial configuration, it registers itself and publishes the device profile in a device registry (as shown in Fig. 3) that enables the clients to discover the devices [13,15].

3.2. Template Layer

In this architectural framework *Device Template* is a prototype that defines configurations and device type specific information. It can apply to automate the initial configuration of multiple devices and reuse the same configuration. This layer provides universal methodologies to access all the physical objects through device templates. An individual template represents its corresponding devices, and it is constrained by the capabilities of that device. In general, IoT devices are running with software or any development framework with minimum processing and storage capacity by deploying lightweight microservices [53]. Therefore, a device template defines the telemetry that a device sends data with correct units, data types, and sensor gateway. Device templates can also be used to test some device functions, such as policy checking, without the requirement for a real device. All the devices are registered through the templates so that the device can be easily discoverable and manageable. At runtime, the system will be able to identify the fault and rectify it through the device template so that the system will have high fault tolerance. Table 1 shows the properties used for making the device template. The *Device Template* includes few properties from *Device Profile* (as discussed in above section) and some additional properties discussed in following sections.

Table 1
List of properties and parameters used for making Device Template.

Properties	Parameters	Responsibilities
Identification	Name Identifier Device Type	Identification of device in a run-time environment.
Context	Location Observable properties Timestamp	It gives the contextual information of the devices.
Accessibility	Public Private	It provides the availability of the devices.
Capability	Commands Attributes	It specifies the ability of the devices.
Dependency	Event dependency Data dependency	It provides the type of dependencies among IoT devices.

(a) *Identification:* Typically, IoT devices have different configuration policies and processing standards which also depend on the device type and manufacturer. The device's name is usually a string of characters and numbers, whereas the identifier (URI) is a sequence of characters that allows the device to integrate with a network, communicate and exchange data with other devices. The *type* of the device is a level associated with the device, which is used to match it to a series or model. For example, a series of temperature sensors associated with a single application and the same communication protocol is considered as one type of device.

(b) *Context:* The *Context* of an IoT device can be a tuple of its *Location, Observable properties*, and *Timestamp*. As the large variety of "things" connect to the internet, the amount of data increases. It needs filters to pull out the data required for location-based service. Therefore, location is the key parameter in the context of IoT devices. Activities can be obtained by comparing a series of time windows with the same observable property on the same feature of interest.

(c) *Accessibility:* The IoT devices are deployed in one or more specific locations, either owned or occupied by the end-user. The devices that are registered on *public* networks will not work on the *private* network. The device registered in a public network allows access to the data by other devices and applications remotely. If the device is registered in a private network, the access is restricted, i.e., the device will be available within that network only.

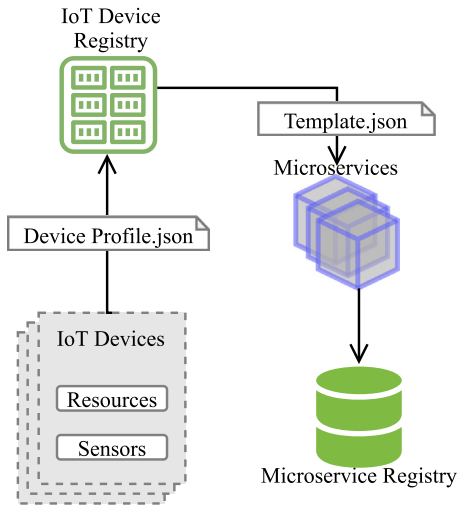


Fig. 3. IoT device registration and microservice creation based on Device Type Templates.

(d) *Capability:* The *Capabilities* of the IoT devices are the core properties, based on which an application interacts with other smart devices. Capability specifies the features of a device through which the user can find a suitable device for a specific application. Capability can be represented as a combination of *Commands* and *Attributes*. *Commands* represent the methods through which the user can control the device, whereas *Attributes* represent state information or properties of the device such as accuracy, range, precision, etc.

(e) *Dependency:* The *Dependency* occurs when the number of types of IoT devices increases. As a result, the number of functionalities also increases. It could provide some complex functionalities such as data fusion and network collaboration, which reflects the dependencies among IoT devices. The *Event dependency* occurs if the independent events are expressed as transactions comprising a single operation. Similarly, the triggering of an event depends on values from multiple sensors causes *Data dependency*.

3.3. Microservice Layer

The *Microservice Layer* is responsible for developing microservices and delivering services based on user needs. The microservice architecture divides functionalities into distinct methods and classes, with each unit responsible for completing the task through the use of microservices. Each device type's template has its own set of functionalities, data formats, and communication protocols. Therefore, a template associated with one or more microservices can operate independently to provide configuration and data delivery services. Here, several microservices can be combined in order to provide an on-demand service in an application.

3.3.1. Building microservices

The IoT devices support extensible plugins, i.e., a set of instructions to handle low-level device communication and provide APIs to microservices. As illustrated in Fig. 5 each device has its own template via which it is represented in the IoT network. For each device type, the Device type Template (DT), Protocol (P), and Data Format (DF) are utilized to create microservices. Fig. 4 depicts three alternative scenarios for constructing microservices: (a) there is one template (DT_i) for one device type (D_i) and single microservice (μs_i using $\langle DT_i, P_i, DF_i \rangle$) is created. (b) multiple microservices μs_i and μs_j can be built using same template having multiple data format $\langle DT_i, P_i, DF_i \rangle$ and $\langle DT_i, P_i, DF_j \rangle$ respectively. For example, the Thermo-hygrometer offers both temperature and humidity in one unit. Although a single template can be used for configuration, microservices for temperature and humidity measurement may differ because of data format differences. (c) a single template (DT_i) is utilized for multiple devices (D_i, D_j) and multiple microservices (μs_i and μs_j using $\langle DT_i, P_i, DF_i \rangle$ and $\langle DT_j, P_j, DF_j \rangle$ respectively) are generated as the device type is same. For example, the sensor that monitors both body temperature and weather temperature can be built using the same template. The type of device and its properties are identical, but the communication protocol may differ. As a result, separate microservices can be built for such devices.

Algorithm 1 Microservice Creation

Input: Device Template $DTemp$ ▷ Input as .json format
Output: Microservice Instances

1: Read ID of Device D_n from Device Template $DTemp_n$
2: **for** each device D_i in Device Registry Reg_{ij} **do**
3: **if** $ID_i = ID_n$ **then**
4: DT_n = getDeviceType(D_n)
5: P_n = getProtocol(D_n)
6: DF_n = getDataFormat(D_n)
7: **if** $(DT_i \neq DT_n) || (P_i \neq P_n) || (DF_i \neq DF_n)$ **then**
8: CreateNewMicroservice($ID_n, DTemp_n$)
9: **else**
10: UpdateMicroservice($ID_n, DTemp_n$)
11: **end if**
12: **else**
13: CreateNewMicroservice($ID_n, DTemp_n$)
14: **end if**
15: **end for**
16: **procedure** CreateNewMicroservice($ID_n, DTemp_n$)
17: Read($DTemp_n$) ▷ $DTemp_n$ is the device profile for D_n
18: **for** each attribute A_j of $DTemp_n$ **do** ▷ Where $1 \leq j \leq M$
19: $Reg_{nj}(D_n) \leftarrow A_j$ ▷ $Reg_{nj}(D_n)$: Registry entry for device D_n
20: **end for**
21: createRestAPI($R(D_n)$)
22: **end procedure**
23: **procedure** UpdateMicroservice($ID_n, DTemp_n$)
24: **for** each attribute A_j in $DTemp_n$ **do**
25: **if** compareAttribute($R_{nj}(D_n), A_j$) **then**
26: $Reg_{nj}(D_n) \leftarrow A_j$
27: **end if**
28: **end for**
29: **end procedure**

The process of building microservices from device templates is depicted in Algorithm 1. To create and update microservices, the device template (in *.json* format) is utilized. Step 1 to 3 checks the device is registered or not. Then, it finds the device type (DT), protocol (P), and data format (DF) of the new device template. Here, one microservice can be created for one type of device, and multiple microservices can be created if the data format and protocol are different as shown in Fig. 4. If any of DT, P, DF differs, the procedure "CreateNewMicroservice($ID_n, DTemp_n$)" will be executed; otherwise, "UpdateMicroservice($ID_n, DTemp_n$)" will be executed to update the microservices. Step 16 to 22 describes the procedure to create a new microservices, i.e., it creates an ID for new device in order to register the device in the device registry (Reg_{ij}), where i represents the device and j represents the corresponding attribute. Then, all the entities are stored and REST controller is used to provide an API for the microservice, allowing users to communicate with them. The procedure "UpdateMicroservice($ID_n, DTemp_n$)" described in step 23 to 29 is used to update the existing microservices only for the devices same DT, P, DF of an IoT device. Further, the complexity of Algorithm 1 is analyzed as the major component is to find the DT, P, DF of each

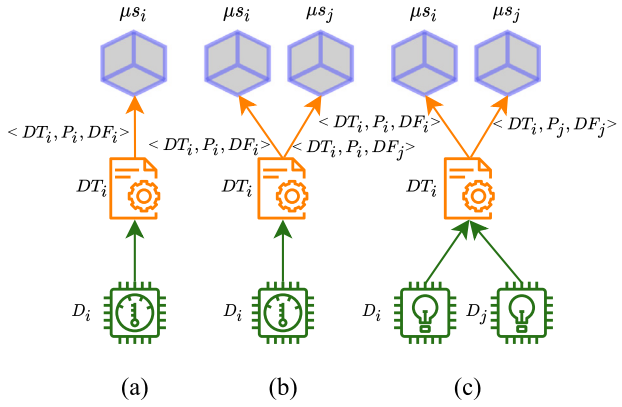


Fig. 4. Different cases of building microservices: (a) One Device Type-One Template-One microservice; (b) One Device Type-One Template-multiple microservices: (c) Same device Type-Same Template-multiple microservices.

device in a device registry. Therefore, the complexity is $O(N \times M)$ where, N is the number of devices registered in device registry and M is the number of attributes.

3.3.2. Microservice registration

During run-time or microservice discovery, clients of service should be aware of the available service instances. Therefore, all the microservice instances need to be registered with the service registry (shown in Fig. 5), which is a database that stores microservice network locations, instances, and addresses. Furthermore, the client can query the service registry for accessing the available service instances.

3.3.3. Data Distribution

The IoT makes sensor data available among applications and devices to enable better services, i.e., exchanging information across devices might result in new and better services. Generally, data distribution is inherited from a pub-sub mechanism in which multiple publishers collect the data from physical devices and publish them to multiple subscribers. In order to avoid the complexity, a Global Data Space (GDS) is used where the publishers/subscribers can read/write data autonomously.

3.3.4. Microservice aggregator

The microservice aggregator maps several microservices according to the topics, where topic represents the category of a device. Afterwards it reduces the search space by selecting an optimal collection of topics and associated microservices based on the user requirements. Each microservice has well-defined interfaces that allow it to communicate and share data with other microservices. So, these microservices can be efficiently designed as a group of containers that can run concurrently and can be spun-up and decommissioned as and when needed. Containerizing microservices helps in efficiently scale the system to meet demands. Here, in this architecture the aggregated microservices are containerized without considering the dependency and design patterns of microservices. Thus, it enables faster execution of user query and significantly enhances reusability that allows users to receive composite information from several microservices.

One of the traditional methods for communicating between microservices is through their REST APIs. However, as the system grows in complexity and the number of microservices increases, communication becomes more difficult, and slowing down the development process. To address the design disadvantage, this architecture aims to decouple producers from consumers, with asynchronous messaging, and uses microservices for both producer and consumers. The producer microservices are responsible for collecting the information from the IoT devices and distributing it across multiple topics. The consumer microservice offers APIs that allow end users to register as a consumer of the application. A topic is a term or category for a stream of data that is published. Consumer microservices subscribe to topics depending on user requests in order to consume data. Let $\mu s = \{\mu s_1, \mu s_2, \ldots, \mu s_{m-1}, \mu s_m\}$ denotes the set of distinct microservices and $T = \{T_1, T_2, \ldots, T_{n-1}, T_n\}$ denotes the set of distinct topics, where $\{m, n\} > 0$. The relation between the topics and microservices is determined using a relation matrix (R). A relation matrix is a logical matrix with binary values (0 and 1), represents the binary relation between microservices and topics. If a microservice (μs) is subscribed to topic (T), the value of R_{ij} is labeled as '1'; otherwise, it is labeled as '0'. The relation matrix is shown in Eq. (1):

$$\mathbf{R} = \begin{array}{c} \\ \mu s_1 \\ \mu s_2 \\ \vdots \\ \mu s_{n-1} \\ \mu s_n \end{array} \begin{array}{c} \begin{array}{ccccc} T_1 & T_2 & \cdots & T_{n-1} & T_n \end{array} \\ \begin{pmatrix} 0 & 1 & \cdots & 1 & 0 \\ 0 & 0 & \cdots & 1 & 0 \\ \vdots & \vdots & \cdots & \vdots & \vdots \\ 1 & 0 & \cdots & 1 & 0 \\ 1 & 0 & \cdots & 0 & 1 \end{pmatrix} \end{array} \text{Where}$$

$$R_{ij} = \begin{cases} 1 & \text{if } T_i \text{ allocated to } \mu s_j \\ 0 & \text{if } T_i \text{ not allocated to } \mu s_j \end{cases} \quad (1)$$

Further, to reduce the latency, this model is built over two functions such as map and reduce as the aggregation policy is based on the observation that there may be idle groups at distinct labels. The workflow of a map reduce technique is as follows: in the first step, the microservices and their related topics are tokenized as <key, value> pairs, i.e, the <keys> are represented as the id of microservices and the <values> are the set of topics assigned to the respective microservices. In the relation matrix (R), each row vector (r) from R is used as input for mapping the topics and microservices. The topics of the non zero elements are taken as <values> for respective microservices. A record for each <key: μs, value:Topics> pairs is recorded in aggregation matrix (A) as shown in step 9 to 12 in Algorithm 2. Following the mapping of each record, the microservices that are subscribing to the same set of topics are combined. The <key, value> pairs are read, one at a time and the <values> of each <key> are compared with the consecutive records. If the <values:Topics> are identical for multiple microservices, then the initial <values:Topics> is retained and the other microservices are mapped to it. In this process, the duplicate <values> and removed from A. Step 13 to 21 in Algorithm 2 shows the reduction process for the technique. The complexity of Algorithm 2 is observed as the main computational part is finding the relation matrix as defined in Eq. (1) and finding the <key, value> pairs and identical topics. Hence, the computational complexity is $O(\mu s \times T)$, where μs is the number of iteration need to check all the microservices and T is the number of iteration used to check all the topics.

3.4. Application Layer

The *Application Layer* is the outermost layer that defines all the applications where the IoT is deployed. IoT applications can be smart homes, smart cities, smart health, animal tracking, etc. It is responsible for delivering application-specific services to users and also responsible for data formatting and presentation. This layer contains components that enable a variety of client interaction interfaces, including web user interfaces, PC clients, and mobile apps, among others. The user's query is sent to the microservice layer by the application. Microservice Identifier looks for service providers or devices that are relevant to the user's query. Then, the application starts to subscribe to the topic, and the data is published by service providers or device microservices.

Algorithm 2 Microservice Aggregation

Input: $(\mu s_1 \dots \mu s_m)$, $(T_1 \dots T_n)$ ▷ T-Topics, μs-Microservice
Output: Aggregation Matrix (A)

procedure μsAggregation$(\mu s, T)$
  for each i of μs **do**
    for each j of T **do**
      if μs subscribe to T **then**
        R$[i][j]$=1 ▷ Find the relation matrix
      end if
    end for
  end for
  for each $row(r)$ of R **do**
    Topic ← Find the index of non zero elements ▷ Find the topics allocated to each μs
    $A[r]$ ← <key:$\mu s[r]$, value:Topic> ▷ Tokenize as <key, value> pairs
  end for
  v=0
  for each $key(k)$ in A **do**
    IdenticalTopic ← value(k).Topic
    if value(v).Topic == IdenticalTopic **then**
      key(v).μs ← IdenticalTopic ▷ Find the identical topics
      Remove the repeated records from A
    end if
    v=v+1
  end for
end procedure

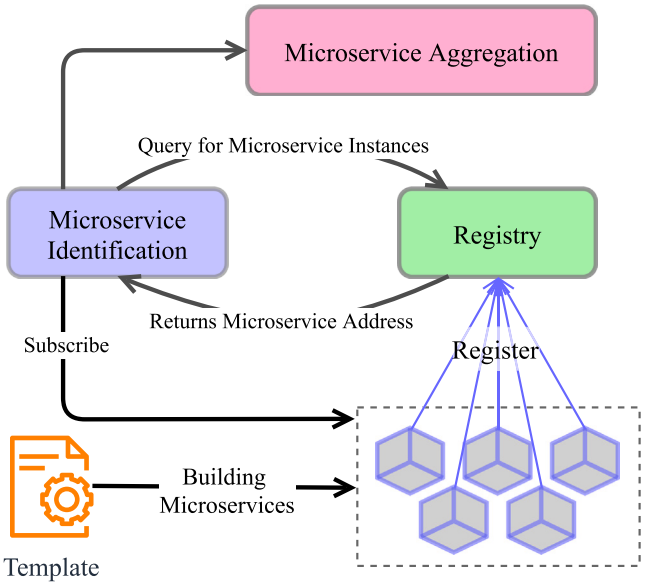


Fig. 5. Block diagram for the process of building, registering, identifying, and aggregating of microservices.

4. Evaluation of architectural components using Event-B

To formally define the proposed architecture and validate its complex behaviors, an Event-B based approach is discussed in this section. In general, IoT system uses event to event coupling therefore, the proposed architectural framework is validated using event-driven approach and described through Event-B language as it is centered on the broad concept of events. Further, Event-B uses incremental modeling, which helps in conceptualizing the system from an abstract specification such as sensors, devices etc., and subsequently introduces more implementation details in device templates and microservices. To validate the system's behavior in a formal and unambiguous manner, it is implemented in "Rodin" platform [54], which supports Event-B notations. Event-B is composed of two components such as *Context* and *Machine*. The static components of the system are represented by sets, constants, and axioms in the *Context*, while the dynamic components are implemented in the *Machine* by variables, invariants, and events. The model's state changes are also represented by some events. The event has guard for condition checking and actions for performing task. The significant concepts and components of the architectural framework such as *Device Profile, Device Template and Microservices,* and *Data Distribution* are validated through Event-B by expressing them as invariants which must be satisfied for all events in the architectural framework.

4.1. Device Profile

This model shows the abstract representation of the sensors and actuators in an IoT system. It only demonstrates the properties and characteristics of IoT devices while interacting with the physical world. The device profile concept is extended from C.K.Rath et al. [24]. The formal description of the deployment, sensing, actuation, and result events is given in Table 2. Whereas the associated context and invariants of the device profile are depicted in Fig. 6 and indicated in italic in the following description of the events.

(a) **Deployment:** This event illustrates how the devices are deployed in a platform. The information necessary during device installation includes the device's unique *identity* for recognition, the *deviceType* and the *platform* to which the device is applicable. The *deviceId* helps in creating the *URI* (Unique Resource Identifier). The action of deploying *DEVICE* into a *platform* is known as *hosting*.
(b) **Sensing:** If the device is a sensor, it will start sensing after being triggered (*Trigger*) by either a *stimulus* or user *input*. The *Event* is the method through which *DEVICE* performs some *Action*. To handle the dependency, the sensor activates the *sensingFlag*, which prevents it from receiving any more input during the event. The *Context* will be the feature of interest (*featureOfInterest*) along with the observable properties (*obsProperty*).
(c) **Actuation:** An *Actuation* carries out a procedure to change the state of the world using an *Actuator*. If *Actuator* is a *DEVICE* and *actuableProperty* is an *Action*, the *Event* will be the *actuator* executing the *actuableProperty*.
(d) **Result:** The *Result* of a Sampling or Actuation is a property that may be used to hold the sample output of an event. The *Output* obtained from the IoT device if the *sensingFlag* is set to *TRUE*. The output is the stream of *sensingData* and *context* with respect to *Time*.

4.2. Device template and microservices

This model formalizes the crucial device and microservice management aspects for the proposed architectural framework. The formal verification of the *Device Template* to *Microservice* creation is represented in this model. The machines demonstrate registration, deregistration, addition, deletion, and replacement of IoT devices. The required context and invariants are described in Fig. 7 and indicated in italic in the following description of the events, whereas the formal representation is listed in Table 3.

(a) **Build Microservice:** Device templates, protocols, and data formats are used to create microservices. If the protocol and data format for a single device differ, different microservices are created as discussed in Section 3.3.1 and it is guarded *buildMicroservice* by *template, protocol,* and *dataFormat*.

Table 2
Machines for device profile.

Events	Guards and actions
(a) Deployment	*grd1*:d ∈ DEVICE *grd2*:uri ∈ Identity *grd3*:p ∈ Platform *act1*:deviceId(d) := uri *act2*:deviceType := {Sensor} ∨ {Actuator} ∨ {EmbeddedDevice} *act3*:hosting(d) := p
(b) Sensing	*grd1*:(i ∈ Input) ∨ (s ∈ Stimulus) *grd2*:i ≠ s *grd3*:sensor ∈ DEVICE *grd4*:sensing ∈ Action *grd5*:obs ∈ obsProperty *grd6*:foi ∈ featureOfInterest *act1*:Trigger := {i ↦ sensor} ∪ {s ↦ sensor} ∪ ϕ *act2*:Event := {sensor ↦ sensing} *act3*:sensingFlag := TRUE *act4*:Context := {obs ↦ foi}
(c) Actuation	*grd1*:actuator ∈ DEVICE *grd2*:actuableProperty ∈ Action *act1*:Event := Event ∪ {actuator ↦ actuableProperty} ∪ ϕ
(d) Result	*grd1*:t ∈ Time *grd2*:data ∈ sensingData *grd3*:sensingFlag = TRUE *grd4*:Context={obs ↦ foi} *act1*:Output := Output ∪ {{data ↦ {obs ↦ foi}}↦ t}

CONTEXT
c0_dp
SETS
DEVICE
PROPERTIES
SYSTEM
USER
CONSTANTS
Sensor
Actuator
EmbeddedDevice
AXIOMS
axm1: partition(DEVICE, {Sensor}, {Actuator}, {EmbeddedDevice})
END

MACHINE m0_dp **SEES** c0_dp
INVARIANTS
inv1: Identity ⊆ PROPERTIES
inv2: deviceType ∈ P(DEVICE)
inv3: deviceId ∈ DEVICE → Identity
inv4: Platform ⊆ SYSTEM
inv5: hosting ∈ DEVICE → Platform
inv6: Input ⊆ USER
inv7: Stimulus ⊆ USER
inv8: (Trigger ∈ Input ↔ DEVICE) ∨ (Trigger ∈ Stimulus ↔ DEVICE)
inv9: Action ⊆ PROPERTIES
inv10: Event ∈ DEVICE ↔ Action
inv11: sensingFlag ∈ BOOL
inv12: (sensingData ⊆ DEVICE) ∨ (Time ⊆ DEVICE)
inv13: (obsProperty ⊆ PROPERTIES) ∨ (featureOfInterest ⊆ PROPERTIES)
inv14: Context ∈ obsProperty ↔ featureOfInterest
inv15: Output ∈ ((sensingData ↔ (obsProperty ↔ featureOfInterest)) ↔ Time)

Fig. 6. Context and invariants for Device Profile.

CONTEXT
c0_dt
SETS
DEVICE (D)
MICROSERVICE (MS)
TEMPLATE
REGISTRY
PROTOCOL
DATAFORMAT
CONSTANTS
ms1, ms2, ms3, ms_n
d1, d2, d3, d_n
AXIOMS
axm1: partition(MICROSERVICE, {ms1}, {ms2}, {ms3}, {ms_n})
axm2: partition(DEVICE, {d1}, {d2}, {d3}, {d_n})
END

MACHINE m0_dt **SEES** c0_dt
INVARIANTS
inv1: microservices ⊆ MICROSERVICE
inv2: devices ∈ microservices → P(DEVICE)
inv3: buildMicroservice ⊆ TEMPLATE × PROTOCOL × DATAFORMAT
inv4: msRegistry ⊆ REGISTRY
inv5: devRegistry ⊆ REGISTRY
inv6: msRegistration ∈ microservices → msRegistry
inv7: devRegistration ∈ (P(DEVICE) ↔ TEMPLATE) → devRegistry

Fig. 7. Context and invariants for Device Template to microservice creation.

(b) **Registration:** Initially, the service instances must be registered with the service registry (*msRegistry*) in order to be discovered and unregistered when unable to handle the request. The microservice will be added to the registry if it has never been registered and is not associated with any device. The *msRegistration* is a procedure for registering microservices that specifies the location of services.

(c) **Deregistration:** When a linked IoT device is deleted, microservices are automatically deregistered from the service registry. If *a_device* is part of *DEVICE (D)*, *a_service* is part of *MICROSERVICE (MS)*, and *a_device* is unavailable or destroyed, the relevant microservice is also deregistered.

(d) **Add Device:** Using a device template, a new IoT device is added to the network. Registering a device involves storing data in the device registry(*devRegistry*) using a template. After registration,

Table 3
Machines for device template to microservice creation.

Events	Guards and actions
(a) Build microservic	*grd1*:template ∈ TEMPLATE *grd2*:protocol ∈ PROTOCOL *grd3*:dataFormat ∈ DATAFORMAT *act1*:buildMicroservice := {template ↦ protocol ↦ dataFormat}
(b) Registration	*grd1*:a_service ∈ MS \ microservices *grd2*:registry ∈ msRegistry *act1*:microservices := microservices ∪ {a_service} *act2*:devices(a_service) := ϕ *act3*:msRegistration ∪ {a_service ↦ registry} :∈ microservices ∪ {a_service} ↦ registry
(c) Deregistration	*grd1*:a_device ∈ D *grd2*:a_service ∈ microservices *grd3*:registry ∈ msRegistry *grd4*:devices = devices \ {a_device} *act1*:microservices := microservices ⩤ {a_service} *act2*:msRegistration \ {a_service ↦ registry} :∈ microservices \ {a_service} ↦ registry
(d) Add device	*grd1*:a_service ∈ microservices *grd2*:a_device ∈ D *grd3*:a_device ∈ D \ union(ran(devices)) *grd4*:template ∈ TEMPLATE *grd5*:registry ∈ devRegistry *act1*:devices(a_service) := devices(a_service) ∪ {a_device} *act2*:devRegistration := devRegistration ∪ {{a_service ↦ template} ↦ registry}
(e) Remove device	*grd1*:a_service ∈ microservices *grd2*:a_device ∈ D *grd3*:a_device ∈ device(a_service) *grd4*:template ∈ TEMPLATE *grd5*:registry ∈ devRegistry *act1*:devices(a_service) := devices(a_service)⩤ {a_device} *act2*:devRegistration := devRegistration ⩤ {{a_service ↦ template}↦ registry}
(f) Replace device	*grd1*:(d1∈D) ∧ (d2∈D) *grd2*:(ms1∈microservices)∧ (ms2∈microservices) *grd3*:template ∈ TEMPLATE *grd4*:devRegistration = devRegistration \ {{d1}↦template} *grd5*:d1 ∈ devices(ms1) *act1*:devices(ms2) := devices(ms2) ∪ {d2} *act2*:devRegistration := devRegistration ∪ {{d2} ↦ template}

each device is assigned to a microservice. The *devRegistry* is a function of every *DEVICE* connected with *TEMPLATE* submitted to registry.

(e) **Remove Device:** When a device is withdrawn from an application, it is deregistered from the *devRegistry* right away. The microservice associated with the device is likewise removed from the service registry. It is expressed mathematically as domain subtraction of *device* and *microservice* from the device and service power set.

(f) **Replace Device:** In the event of a device replacement, the type of device must be the same as the replaced device, i.e., the device template must be the same for both devices. The device template will be used to update the device registry and assign a microservice to the new device.

4.3. Data Distribution

This model explains the communication and data distribution among microservices. This architecture employs the Kafka to consume and deliver data for streaming analytics and data integration pipelines. Producers deliver events to the topic without considering about how or when these event will be handled. The topic then distributes events to any services that must respond to them. This asynchronous integration allows producers to wait for consumers to get data, increasing the system's flexibility. The required context and invariants are described in Fig. 8 and indicated in italic in the following description of the associated events, whereas the formal representation is listed in Table 4.

(a) **Subscribe Topic:** Devices are acting as producers which send messages to a designated resource called a *topic*. The stream of messages from a single, specific topic (*top*) that will be supplied to the subscribing *consumer* in a specific time period is known as a *topicOfInterest*.

(b) **Unsubscribe Topic:** The proposed system often requires to process a large volume of data and needs to distribute the workload during runtime. Thus, there is a need to unsubscribe the *topic* or a group of topics together. To perform this task the *topicOfInterest* of a consumer is simply destroyed or deallocated.

(c) **Send msg:** The messages are sent to the topic for a set time period by the producers. The message sent to the subject is *topicMsg*, while the message sent by the producer is *producerMsg*. The *sessionTime* specifies how long the data must be sent to the subject. Several producers can transmit data to numerous topic at the same time.

(d) **Receive msg:** Consumer receive the message from the subscribed topic. The *msgQueue* is the streamline of events to be served by the consumer. The *consumerMsg* is the data read by the consumer. The consumer able to receive the *topicMsg* if it is subscribed to *topicOfInterest*.

The concept of stream processing is used to process and analyze data contained in Kafka. A stream processor is a processing step that

CONTEXT
c0_ms
SETS
PRODUCER
CONSUMER
MESSAGE
TOPIC
CLUSTER
PROCESSOR

MACHINE m0_ms **SEES** c0_ms
INVARIANTS
inv1: topicMsg ∈ MESSAGE → TOPIC
inv2: producerMsg ∈ MESSAGE → PRODUCER
inv3: consumerMsg ∈ CONSUMER → P(MESSAGE)
inv4: Time ∈ N1
inv5: sessionTime ∈ MESSAGE → (1 .. Time)
inv6: topicOfInterest ∈ (1 .. Time) → (TOPIC ↔ CONSUMER)
inv7: msgQueue ∈ MESSAGE ↔ CONSUMER

Fig. 8. Context and invariants for data distribution.

Table 4
Machines for data distribution.

Events	Guards and actions
(a) Subscribe topic	*grd1:*cons ∈ CONSUMER *grd2:*top ∈ TOPIC *act1:*topicOfInterest(Time) := topicOfInterest(Time) ∪ {top ↦ cons}
(b) Unsubscribe topic	*grd1:*cons ∈ CONSUMER *grd2:*top ∈ TOPIC *grd3:*top ↦ cons ∈ topicOfInterest(Time) *act1:*topicOfInterest := topicOfInterest ∪ {Time ↦ (topicOfInterest(Time) \ {top ↦ cons})}
(c) Send message	*grd1:*prod ∈ PRODUCER *grd2:*top ∈ TOPIC *grd3:*msg ∈ MESSAGE *act1:*topicMsg(msg) := top *act2:*producerMsg(msg) := prod *act3:*sessionTime(msg) := Time
(d) Receive message	*grd1:*cons ∈ CONSUMER *grd2:*msg ∈ MESSAGE *grd3:*msg ↦ cons ∈ msgQueue *grd4:*topicMsg(msg) ↦ cons ∈ topicOfInterest(Time) *act1:*msgQueue := msgQueue \ {msg ↦ cons} *act2:*consumerMsg(cons) := consumerMsg(cons) ∪ {msg}

transforms data in streams by accepting one input record at a time from producers and producing one or more output records for consumers. Consumers can retrieve messages by topic, partition, and offset using a Kafka broker. The refined model of *Data Distribution* and associated events are defined in Fig. 9 and Table 5 respectively.

(a) **Publish Data *refines* Send msg:** A *Broker* takes messages from producers and stores them in a unique offset keyed format in a database. The message transmitted from *producer* to *broker* is known as *PushMsg*. Similarly, *InputStream* is the streaming of data from a *producer* to *StreamProcessor*.
(b) **Subscribe Data *refines* Receive msg:** Complex operations (aggregation, composition, etc.) can be applied to multiple input streams and multiple records in stream processing. This system stores the messages for a configurable length of time since Kafka employs a pull-based model. The action of *consumers* pulling data from the *broker* is *PullMsg*, while the action of aggregating events for consumers is *OutputStream*.

Table 5
Refined machines for data distribution.

Events	Guards and actions
(a) Publish data (*refines* Send Message)	*grd4:*br ∈ Broker *grd5:*sp ∈ StreamProcessor *act5:*PushMsg := {prod ↦ br} *act6:*InputStream := {br ↦ sp}
(b) Subscribe data (*refines* Receive Message)	*grd4:*br ∈ Broker *grd5:*sp ∈ StreamProcessor *grd6:*aggregator ∈ PROCESSOR *act5:*PullMsg := {br ↦ cons} *act6:*OutputStream := {{sp ↦ aggregator} ↦ cons}

4.4. Validation of formal model

Here, the formal model is validated using the proof obligation guidelines available in Rodin platform, which serve as a foundation for determining the correctness of Event-B specifications. Proof obligations(POs) include the correctness of predicates and expressions in invariants, guards, actions, and witnesses for all events, feasibility checks, variable reuse checks, and guard strengthening [25]. The POs are automatically generated by Rodin platform tool. Various POs such as invariant preservation (INV), well-definedness of an action (WD), action simulation (SIM), etc are need to be satisfied for validation of the model. The proof statistics for this model is shown in Table 6. The tool generates 60 POs for all the machines described in Section 4.1 to 4.3, out of which 36 POs are Invariant preservation PO rules (INV), 9 POs are Well definedness of guard PO rules (WD), and 15 are Action simulation PO rules (SIM). Rodin platform automatically discharges all POs from each machine. Hence, it is proved that each machine's invariants, guards, and actions are defined properly and correct by its construction.

5. Experimental results and discussion

This section explains how the benefits of the microservice-driven IoT architecture are realized by implementing a case study.

5.1. Implementation with case study

The devised architectural framework is primarily implemented to simulate using the *Spring Boot* application and *Kafka*, along with some other components such as H2-database [55], which helps in realizing

MACHINE m1_ms **REFINES** m0_ms **SEES** c0_ms
INVARIANTS
inv1: Broker ⊆ CLUSTER
inv2: Partition ⊆ CLUSTER
inv3: StreamProcessor ⊆ PROCESSOR
inv4: database ⊆ PROCESSOR
inv5: Aggregator ⊆ PROCESSOR
inv6: InputStream ⊆ PRODUCER → StreamProcessor
inv7: OutputStream ⊆ (StreamProcessor → Aggregator) → CONSUMER
inv8: PushMsg ∈ PUBLISHER ↔ Broker
inv9: PullMsg ∈ Broker ↔ SUBSCRIBER

Fig. 9. Invariants for refined model of data distribution.

Table 6
Proof statistics using Rodin.

Machine	Events	Invariants	Proof Obligations (POs)
m0_dp	4	15	15
m0_dt	6	7	19
m0_ms	4	7	14
m1_ms	2	9	12
Total	**16**	**38**	**60**

Table 7
System configuration used to perform the experiment.

Components	Description
Operating system	Name: Ubuntu 20.01.3 LTS Type: 64-bit Processor: Intel Xeon(R) CPU E5-1660 v3 @ 3.00 GHz
Programming language	Java-openjdk version "1.8.0_292"
Microservice	SpringBoot-1.5.4.0.RELEASE
Registry	H2-Database
Kafka	Apache Kafka-2.13-3.0.0
APIs Platform	Postman

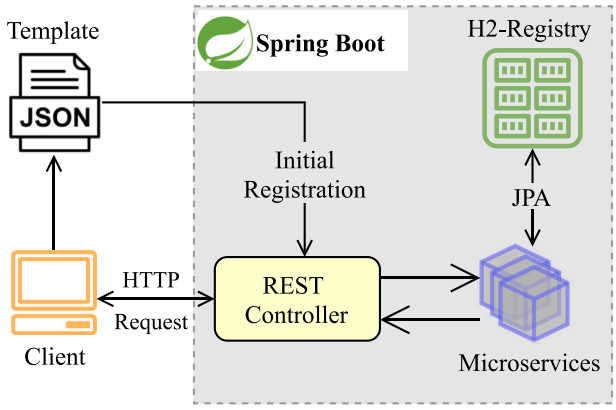


Fig. 10. Template to microservice creation.

some crucial components of the architecture. A Smart City application is considered as a case study. In a smart city, travelers can use a Smart Traffic Management System to find a convenient route to avoid traffic congestion. This system necessitates the handling of large data streams from many environments monitoring IoT devices located in different places, such as traffic posts, industrial locations, local automobiles, and street lights, among others. In this case study, the real-time operations starting from the Device Layer to Application Layer are demonstrated, including device registration, template creation, microservice creation, and microservice aggregation. The implementation of the proposed model is comprised of two main components: *Device Template and Microservice Creation* as well as *Microservice Aggregation*. The whole model is implemented in Java programming language and deployed in the Ubuntu platform. Spring boot application is used for setting up a microservice system, and H2-database is used as a registry. This model also adopts Kafka-centric microservice system to handle the data streams. Further, the microservices are communicating with each other using Kafka as an intermediary. Table 7 lists the system configurations and tools used to implement the case study.

In this simulated case study, the IoT devices have a number of configuration policies, and it contains a significant amount of metadata. The simulated IoT devices are registered through the *Device Template* for easy configuration and to perform their activities with the intended microservices. These devices are then classified according to their type, and templates are created based on this classification. For example, a temperature sensor using the "LoRa" communication protocol is deployed at traffic posts to monitor the temperature of that specific area. Another temperature sensor uses the "Wi-Fi" protocol and is deployed in a smart home setting. Both devices are configured using a single device template because they are the same type of device, but they are accessed via two distinct microservices because their applicability and communication protocols are different. Initially, the participating IoT devices are registered in a device registry by providing the device profile information such as ID, Name, Category, Context, etc., as depicted in Fig. 2. Then the templates are created in JSON format based on device types (shown in Fig. 11). Further, creating microservices application in *"Spring Boot"* includes various specialized components such as *'@RestController', '@Entity', '@Service', '@Repository'*, etc (here the standard spring boot notations are used) [56]. The procedure for creation of the microservices from the device template is described in Algorithm 1.

The annotation *'@RestController'* indicates that the class is a controller. It is a specialized component that handles API mappings (GET, POST, DELETE, etc.) and the response body. The *'@Entity'* specified the class as an entity and mapped it to a database table. The *'@Service'* annotation is used with classes that provide some functionalities. Similarly, the *'@Repository'* annotation emulates a collection of items by encapsulating storage, retrieval, and search functionality. Microservices are developed from the device template by merging all of the above specialized components. The *'@Entity'* class receives the file 'template.json' as input, which parses the JSON objects and maps them to the database table. The class annotated with *'@Repository'* defines all the records that are kept in the H2-database. The API mappings for the services are created in the *'@RestController'* annotated class, and the functions are described in the *'@Service'* annotated class, which acts as the REST API response body. Fig. 10 displays the microservice creation process in the Spring Boot application, whereas Fig. 12 depicts the microservices registry.

The created microservices are then aggregated based on user demand by using *"Apache Kafka"* which makes it simple to operate, maintain, and monitor. Kafka is a messaging system that allows microservices to interact with one another. In this case study, the producers of Kafka are IoT devices and their associated microservices, whereas consumers are cloud storage and atomic microservices. Each device

```
{
  "deviceType": "Temperature Sensor",
  "observableProperties":"Temperature",
  "accessibility":"public",
  "applicability": "Environment temperature measure",
  "dependency": true,
  "battery": false,
  "dataDistribution": "Global",
  "communicationProtocol": "RS-232",
  "storage": "cloud",
  "dataType": "String",
  "device": {
   "deviceName": "E8PC",
   "gatewayAddress": null,
   "memory": null,
   "timestamp":"02/10/21 15:06",
   "powerSupply": "12v",
   "geo": {
    "lat": "-37.3159",
    "lng": "81.1496"
   }
  }
}
```


Fig. 11. JSON file of the device type template response to create microservice.

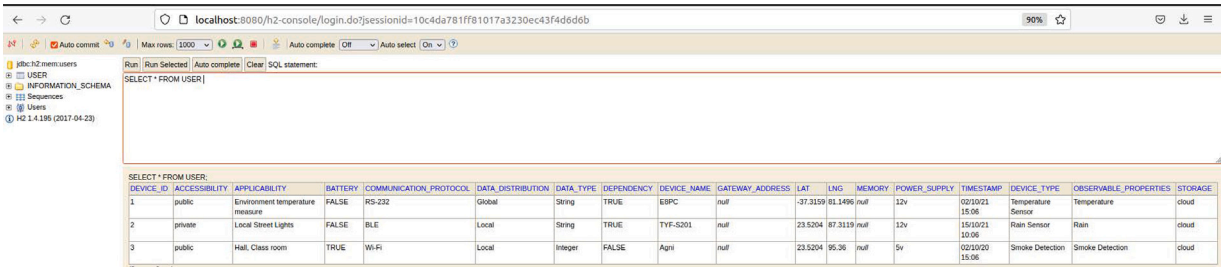


Fig. 12. Snapshot of the sample registry used in this architecture.

sends data to a separate Kafka topic; no two producers send data to the same Kafka topic. Atomic microservices ingest the data from the Kafka topics on the consumer side. Microservices are then registered with a specified group-id to Kafka [57] while consuming data from Kafka.

For the illustration of the case study, ten microservices ($\mu s_1, \ldots, \mu s_{10}$) and five topics ($T_1, \ldots, T_5$) are considered. The topics are named as Temperature (T_1), Humidity(T_2), Gas(T_3), Traffic Volume (T_4), and Crowd (T_5). The IoT devices are sending data to respective topics through the producer microservices. In the consumer side, each microservice belongs to distinct groups to consume the data from different topics (T). The consumers join a group by using the group-id. As depicted in Fig. 13, the μs_1 is subscribed to topics T_1, T_2, and T_5 by maintaining three distinct groups. Similarly, T_1, T_2, T_3 and T_5 are subscribed by the μs_2 in four different groups. The rest of the microservices are grouped together and subscribed to the topics based on its requirements. However, they cannot subscribe to multiple topics by using a single group-ids because other microservices present in the group may not require numerous topics. The observation is that a single microservice needs to maintain many group-ids to consume data from several topics, i.e., if a user desires data on temperature and traffic volume, it must subscribe to two topics by joining two different groups. The Microservice Aggregation is employed to reduce latency and the high degree of occurrence as well as increase the reusability. It collects the topics depending on user requirements and makes them available to a specific user as a service. Algorithm 2 described in Section 3.3.4 is used for microservice aggregation. The topics(T) and microservices(μs) are taken as input to the procedure μsAggregation(). As shown in Fig. 13, five microservice groups are taken that are subscribed to five different topics. The subscriptions of topics by individual microservices are represented by the relation matrix in Fig. 14. The matrix is then hashed row by row in the form of <key, value> pairs. The index of the row is used as key element while non zero elements for individual rows are considered as values, resulting in a row vector of (key, [values]) as depicted in Fig. 14. The records are then sorted and shuffled in a grouping procedure. Duplicate records are eliminated by grouping keys with the same values together. This procedure is repeated until all of the records have been shuffled. Finally, the unique values having a group of topics are listed and assigned to respective microservices.

5.2. Analysis of the results

To evaluate the performance, two parameters are considered, i.e., execution time(ET) and Frequency of subscription (f). Execution time is the total time required to allocate the topics to the microservices. The execution time is collected in the system through the time difference between the time for sending the first and last request to subscribe to the topic. If the $StartTime()$ is captured at the first subscription of microservice to the topic and $StopTime()$ is the time captured at the last subscription. Then, the total execution time (ET) is:

$$ET = StopTime() - StartTime(); \quad (2)$$

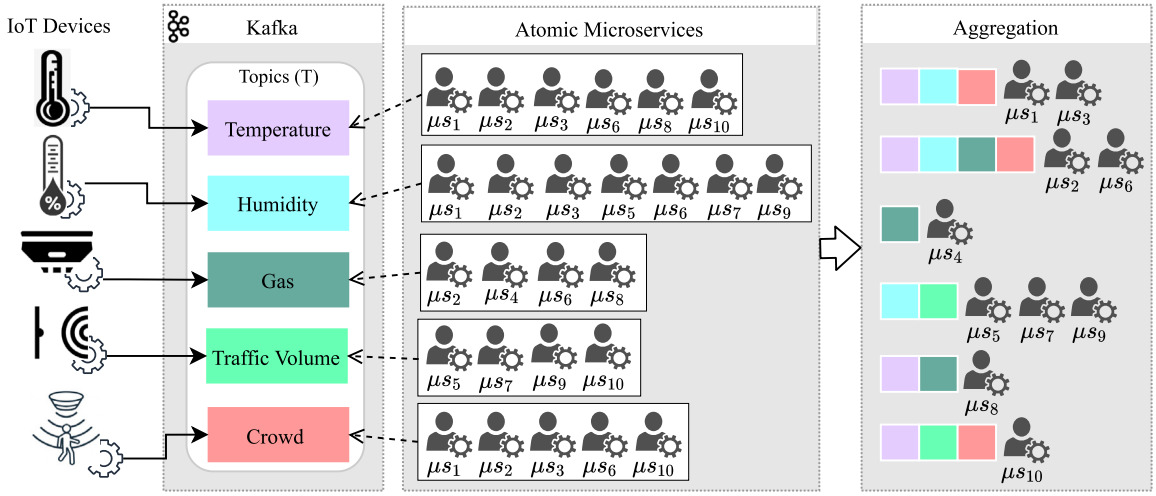
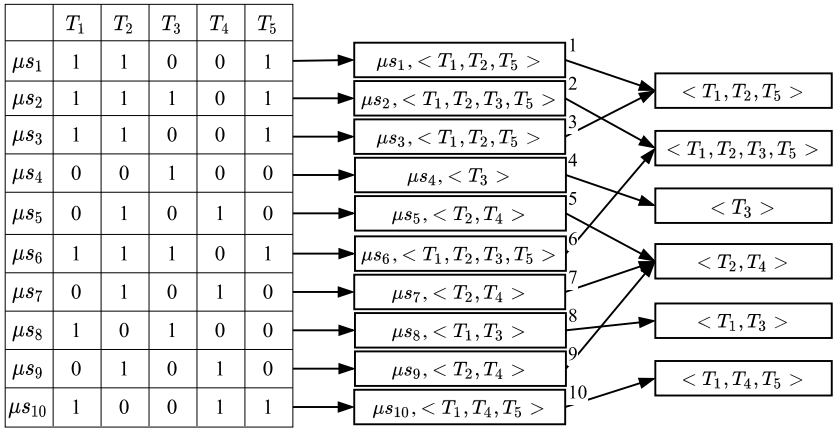


Fig. 13. Internal structure of Kafka and topic aggregation.



	T_1	T_2	T_3	T_4	T_5
μs_1	1	1	0	0	1
μs_2	1	1	1	0	1
μs_3	1	1	0	0	1
μs_4	0	0	1	0	0
μs_5	0	1	0	1	0
μs_6	1	1	1	0	1
μs_7	0	1	0	1	0
μs_8	1	0	1	0	0
μs_9	0	1	0	1	0
μs_{10}	1	0	0	1	1

Fig. 14. Illustration of aggregation technique: records and resultant topics are aggregated using the relation matrix (R) as input.

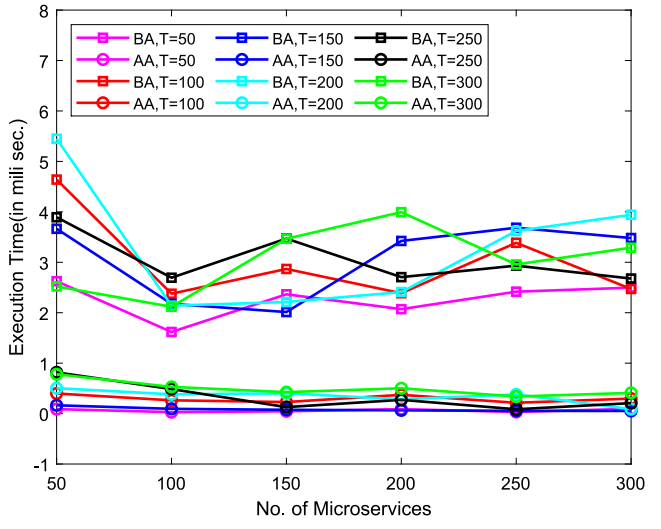


Fig. 15. Execution time of before aggregation (BA) and after aggregation (AA) with the variation of microservices and topics (T).

The variation in execution time before and after aggregation (BA-Before Aggregation, AA-After Aggregation) is shown in Fig. 15. The experiments are performed by taking the number of Topics and Microservices as 50, 100, 150, 200, 250, and 300. The number of Topics and Microservices beyond 300 couldn't be performed due to system configuration constraints. It can be clearly observed that in Fig. 15, the execution time after aggregation is comparatively less than before aggregation at any scale of both Topics and Microservices. This is because, before aggregation, all the constituent microservices are accessed individually through their respective interfaces; this increases the number of interactions which leads to inconsistent and increased execution time. Whereas, after aggregation, multiple topics are combined together as a single entity and accessed through an interface, i.e., the execution time is reduced as the reuse of microservices increases. To understand this reduction in execution time we conducted a series of experiments with varying number of microservices and topics. The results are depicted in Fig. 16, where the concept of frequency is used, which measures per user subscription. If $(T_1, T_2, \ldots, T_n)$ is the number of topics subscribed by a microservice (μs), then the frequency of the microservice is summation of number of subscriptions of the topics necessary to meet the user demand, i.e., $\sum_{i=1}^{n} T_i$, where, n is the number of topics. Then, the total $Frequency(f)$ is the summation of frequency of all constituent microservices. It is expressed in the following Eq. (3).

$$Frequency\ (f) = \sum_{j=1}^{m} \mu s_j \left(\sum_{i=1}^{n} T_i \right), \text{Where } m = \text{number of microservices.} \tag{3}$$

The frequency value comparison of before aggregation and after aggregation are shown in Fig. 16(a) and (b) respectively by varying

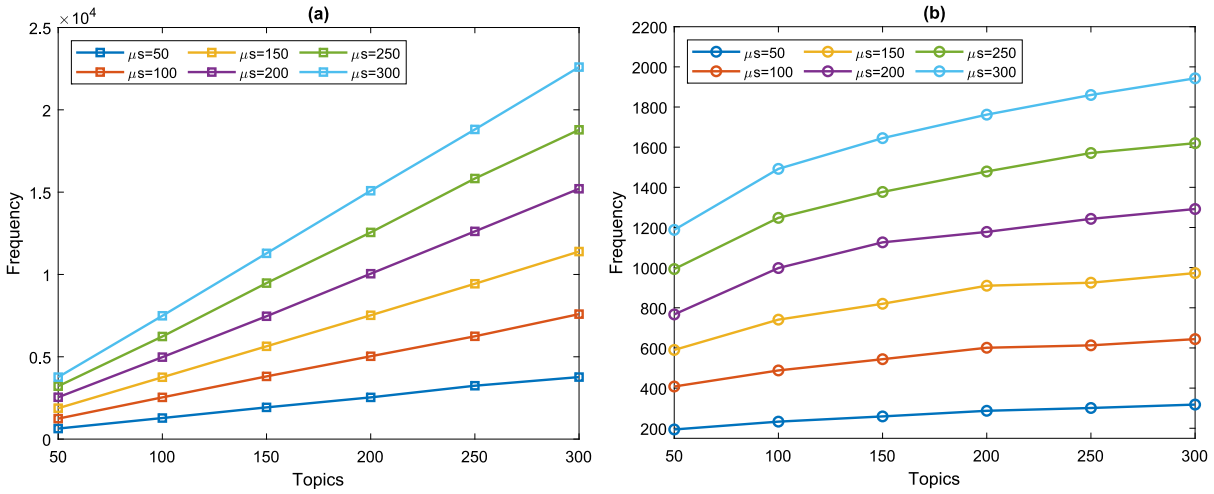


Fig. 16. Frequency analysis with the variation of both microservices and topics; (a) Frequency before aggregation, (b) Frequency after aggregation.

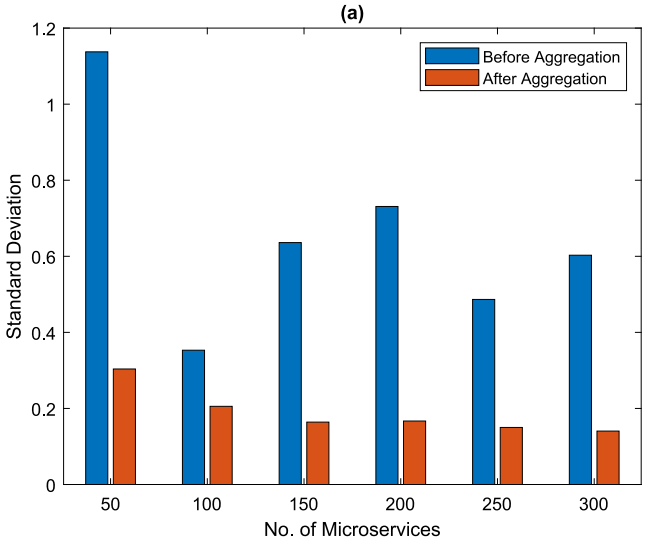


Fig. 17. Standard deviation of execution time.

both the topics and microservices. It is observed that the frequency after aggregation is found to be around ten times lower than the frequency before aggregation. If we consider topic variation from 50 to 300 at a given number of microservices of 200, the frequency before aggregation is (2541, 4978, 7462, 10 045, 12 617, 15 205), which is subsequently decreased to (767, 998, 1126, 1178, 1243, 1292) after aggregation. A similar pattern can be observed for other values of microservices. This is because the aggregation mechanism described in Algorithm 2 eliminates the repeated combination as discussed in Section 3.3.4. This reduction in frequency after aggregation improves the latency and higher degree of topic reuse.

To demonstrate the scalability of the proposed approach, a series of experiments has been carried out, and the results are shown in Fig. 17 which shows the Standard Deviation (SD) of execution time (ET). It shows the degree of stability in execution time with the variation of microservices and topics. Let us consider, $(x_1, x_2, \ldots, x_n)$ are the execution time captured with the variation of number of topics, where N is the number of observations. The standard deviation can be calculated as:

$$SD = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}, \text{Where } \mu = \frac{1}{N}(x_1 + x_2 + \cdots + x_n) \tag{4}$$

Here, a low deviation value represents better consistency in terms of execution time. The results in Fig. 17 show a high deviation of execution time before the aggregation, whereas it becomes significantly less after aggregation. The deviation of execution time also becomes saturated with the increase of microservices and topics, which indicates the scalability of the system.

Again, in order to evaluate the behavior of the system with respect to the change in number of topics and microservices some experiments are carried out. In the proposed architectural framework this change may occur while adding or removing devices from the system. To observe the system's behavior, Fig. 18 presents the performance results under varied argument settings by calculating the percentage of aggregation among the present microservices. Higher aggregation percentage indicates the high level of reuse. The experiments are carried out in with respect to the cases: (a) Increasing number of microservices from 50 to 300 while fixing the number of topics at 200; (b) Increasing the number of topics while fixing the number of microservices at 200; (c) Increasing both the topics and microservices from 50 to 300. When the number of topics is fixed, and the number of microservices is increased (as shown in Fig. 18(a)), there is an optimal aggregation percentage, i.e., approximately 89%. This is because the combination of topics tends to saturate at one point as the total number of topics remains unaltered. Again, when the number of topics are increased (shown in Fig. 18(b)

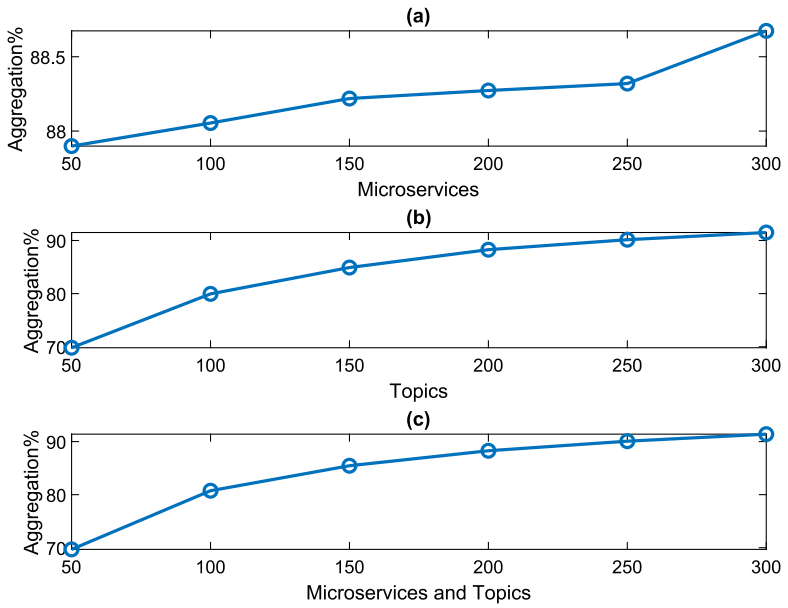


Fig. 18. The performance of results with aggregated percentage; (a) Variation of microservices with topic = 200; (b) Variation of topics with microservice = 200; (c) Variation of both microservices and topics.

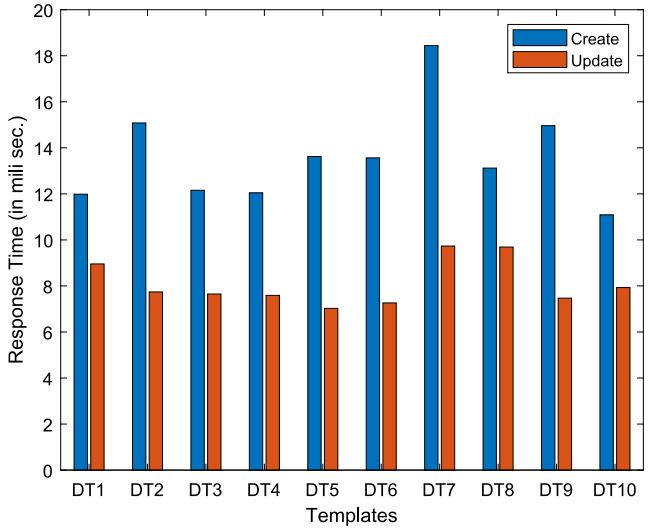


Fig. 19. Response time for creating and updating microservices using different templates.

& (c)), there is a significant performance improvement (i.e., approximately 92%). It is because there are numerous topic combinations available for aggregation. Thus, the aggregation percentage increases with the increasing number of topics in the system. It shows that the proportion of reuse is significant in terms of devices, but it remains constant as the number of users increases as the aggregation based on topics takes place.

In the proposed architectural framework, updating service is a crucial task. To evaluate the efficiency, experiments are carried out to check the Response Time (RT) for creating and updating microservices. The result is depicted in Fig. 19. Here, the response time is calculated as the time difference between the state of request to create or update the microservices ($RequestTime()$) and the state of completion of the request ($CompleteRequestTime()$), i.e.,

$$RT = CompleteRequestTime() - RequestTime() \quad (5)$$

The Response time is taken from the "Postman" tool which is an API platform for building and managing APIs. In this tool, the response time is broken down into various instances such as Socket initialization, DNS lookup, TCP handshake, Transfer start, and Download. Creating a template takes instances of the response time while DNS lookup and TCP handshake time is reduced while updating the microservices. For demonstration, ten different templates (DT1, DT2, ..., DT10) of different sizes are taken to analyze the response time. The Response time is captured by providing a template in JSON format to create microservices and update the microservices by changing certain parameters in the JSON file. All the parameters are not necessarily to be changed while updating the device template. It indicates the flexibility and control over deployments of both IoT devices and microservices. Therefore, this set of tests shows that new devices can be easily added to an existing solution from the template. Further, updating the existing services linked to any IoT device is faster than creating new ones. In addition to this, as discussed in Section 3.3.1, the microservices enable seamless interactions among heterogeneous devices, these features make it highly interoperable.

5.3. Comparative analysis

In this section a comparative analysis is carried out between the proposed mechanism and two related works namely, Service Aggregation Queuing Delay (SAQD) [17], and Docker Layer placement for On-Demand services (DLPOD) [52]. Here, the service aggregation method of the proposed mechanism, SAQD [17] and DLPOD [52] is implemented in the same experimental environment as described in Section 5.1 to compare the performance, scalability and reusability aspects.

The performance of the proposed mechanism, SAQD and DLPOD is evaluated and compared with respect to the execution times by using an increasing number of microservices. Fig. 20 shows performance comparison in terms of time to aggregate the microservices. Here, the execution time and standard deviations are calculated using Eqs. (2) and (4). From Fig. 20(a) it can be observed that the proposed model executes faster than SAQD [17] and DLPOD [52] with increasing no of microservices. Whereas, in Fig. 20(b) it can be observed that the standard deviation of execution time for the proposed model reduces with increasing number of microservices. The similar pattern can be observed with the other two approaches as well however, the value

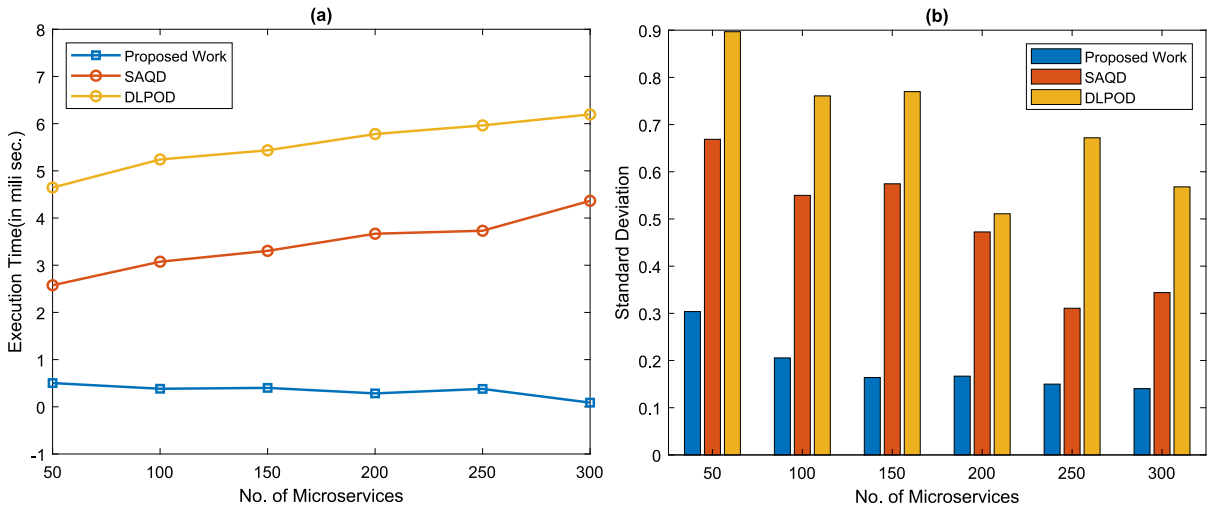


Fig. 20. Performance of various aggregation model by varying number of microservices. (a) Execution time with increase of no. of microservices; (b) Standard deviation of execution time with increase of no. of microservices.

of the standard deviation of the proposed approach is significantly lower compared to the SAQD and DLPOD. This is because of the fact that SAQD uses queuing time to conduct service aggregation at each hop. In addition, the differentiation threshold is continuously changed and determined based on the length of the queue prior to aggregation which results a long execution time. Whereas, DLPOD uses integer linear programming to solve all service serialization (ILP-AS) problem which take longer to execute since they consume more memory during each run. Here, integers represent a set of possibilities where finding only integer is more difficult being more particular type of solution. Therefore, it requires longer time to aggregate the services and difficult to scale.

Again to compare reusability aspects of the proposed model, SAQD and DLPOD the frequency difference metrics is considered as described in Eq. (3). The experimental results are plotted in Fig. 21. For the proposed model, the frequency difference between after and before aggregation is shown, whereas for SAQD and DLPOD, requested service frequency and service reusability frequency are taken into consideration. Fig. 21 shows that the proposed work has a reasonably high frequency compared to the other two methods. Increase in frequency indicates higher reusability. It also observed that in proposed work and DLPOD, the frequency reusability gradually increases with the increase of number of microservices whereas it remains same in SAQD. This is because of the fact that SAQD enables routers to locally aggregate data for services, which reduces the quantity of data. In DLPOD the frequency of reusability increases with the number of microservices, but it is relatively low compared to the proposed model. Because, DLPOD employs sorted queue method, which may be advantageous for the heuristic to prioritize larger services to aggregate but reduces the overall reusability.

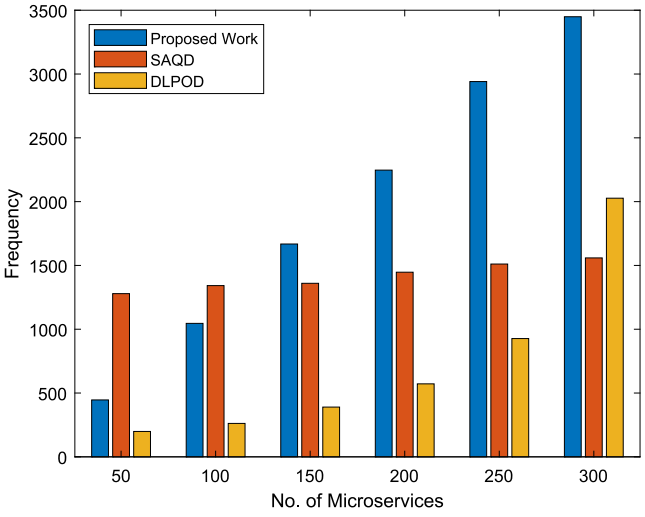


Fig. 21. Frequency difference with varying no. of microservices.

6. Conclusion

This paper proposed a scalable and interoperable microservice-based IoT architecture comprised of four layers where device profile and template for device type provide a generic framework to register the IoT devices and create microservices. The aggregation of microservices is based on the application-specific information, where both the device and the microservices are reused. In addition, each microservice is used in conjunction with others to deliver on-demand services which scale efficiently. Here, the creation of microservice from device templates enabled ease of adding new devices or updating an existing service. Again, the microservices hide the complex data exchange mechanisms among the devices by providing a simple interface and enabling seamless interaction among the heterogeneous devices which makes the devised architectural framework highly interoperable. These crucial characteristics are then formally validated using the Event-B modeling language in the Rodin platform with the help of proof obligations. Further, the benefits of the devised microservice-driven IoT architecture are realized by implementing a case study. From the experimental results, it is observed that the aggregation mechanism of the devised architectural framework significantly reduces the latency of the microservices. The experimental results also shows that the systems scale efficiently with the increase in the number of microservices. Again, it is also evident that the devised architectural framework also offers a high level of reusability of the microservices. In addition, the experimental results depicts that the device templates used in the devised architectural framework not only enhance the integration and interoperability but it also enables the easy update of devices and corresponding microservices. Therefore the devised architectural framework can manage a large number of independent and reusable microservices which provide sufficient scalability and interoperability. Finally the proposed model is compared with two other similar approaches. The obtained result shows that the proposed architectural framework significantly reduces the latency after aggregation, supports efficient scaling and reuse of services.

The future work includes extending the proposed methodology to cater to the containerized microservice and addressing different issues related discovery and scheduling of services for the IoT-based application framework. Further, creating a secure data distribution mechanism is also an important task.

CRediT authorship contribution statement

Chouhan Kumar Rath: Investigation, Methodology, Resources, Writing – original draft. **Amit Kr. Mandal:** Writing – review and editing, Supervision. **Anirban Sarkar:** Conceptualization, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] B. Mayer, R. Weinreich, An approach to extract the architecture of microservice-based software systems, in: 2018 IEEE Symposium on Service-Oriented System Engineering (SOSE), IEEE, 2018, pp. 21–30, http://dx.doi.org/10.1109/SOSE.2018.00012.

[2] M. Baboi, A. Iftene, D. Gîfu, Dynamic microservices to create scalable and fault tolerance architecture, Procedia Comput. Sci. 159 (2019) 1035–1044, http://dx.doi.org/10.1016/j.procs.2019.09.271, Knowledge-Based and Intelligent Information & Engineering Systems: Proceedings of the 23rd International Conference KES2019.

[3] S. Pulparambil, Y. Baghdadi, Service oriented architecture maturity models: A systematic literature review, Comput. Stand. Interfaces 61 (2019) 65–76.

[4] L. Alonso, J. Barbarán, J. Chen, M. Díaz, L. Llopis, B. Rubio, Middleware and communication technologies for structural health monitoring of critical infrastructures: A survey, Comput. Stand. Interfaces 56 (2018) 83–100.

[5] A. Javed, A. Malhi, T. Kinnunen, K. Främling, Scalable IoT platform for heterogeneous devices in smart environments, IEEE Access 8 (2020) 211973–211985, http://dx.doi.org/10.1109/ACCESS.2020.3039368.

[6] C. Santana, L. Andrade, F.C. Delicato, C. Prazeres, Increasing the availability of IoT applications with reactive microservices, Serv. Orient. Comput. Appl. 15 (2) (2021) 109–126, http://dx.doi.org/10.1007/s11761-020-00308-8.

[7] C. Lai, F. Boi, A. Buschettu, R. Caboni, IoT and microservice architecture for multimobility in a smart city, in: 2019 7th International Conference on Future Internet of Things and Cloud (FiCloud), 2019, pp. 238–242, http://dx.doi.org/10.1109/FiCloud.2019.00040.

[8] T. Cerny, Aspect-oriented challenges in system integration with microservices, SOA and IoT, Enterpr. Inf. Syst. 13 (4) (2019) 467–489, http://dx.doi.org/10.1080/17517575.2018.1462406.

[9] N. Dragoni, S. Giallorenzo, A.L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: yesterday, today, and tomorrow, Present Ulterior Softw. Eng. (2017) 195–216, http://dx.doi.org/10.1007/978-3-319-67425-4_12.

[10] G. Cherradi, A. El Bouziri, A. Boulmakoul, K. Zeitouni, Real-time hazmat environmental information system: A micro-service based architecture, Procedia Comput. Sci. 109 (2017) 982–987, http://dx.doi.org/10.1016/j.procs.2017.05.457.

[11] S. Trilles, A. González-Pérez, J. Huerta, An IoT platform based on microservices and serverless paradigms for smart farming purposes, Sensors 20 (8) (2020) 2418, http://dx.doi.org/10.3390/s20082418.

[12] A.d.M. Del Esposte, E.F. Santana, L. Kanashiro, F.M. Costa, K.R. Braghetto, N. Lago, F. Kon, Design and evaluation of a scalable smart city software platform with large-scale simulations, Future Gener. Comput. Syst. 93 (2019) 427–441, http://dx.doi.org/10.1016/j.future.2018.10.026.

[13] W. Jin, D. Kim, Resource management based on OCF for device self-registration and status detection in IoT networks, Electronics 8 (3) (2019) 311, http://dx.doi.org/10.3390/electronics8030311.

[14] S.K. Roy, S. Misra, N.S. Raghuwanshi, SensPnP: Seamless integration of heterogeneous sensors with IoT devices, IEEE Trans. Consum. Electron. 65 (2) (2019) 205–214, http://dx.doi.org/10.1109/TCE.2019.2903351.

[15] L. Sun, Y. Li, R.A. Memon, An open IoT framework based on microservices architecture, China Commun. 14 (2) (2017) 154–162, http://dx.doi.org/10.1109/CC.2017.7868163.

[16] M. Noura, M. Atiquzzaman, M. Gaedke, Interoperability in internet of things: Taxonomies and open challenges, Mob. Netw. Appl. 24 (3) (2019) 796–809, http://dx.doi.org/10.1007/s11036-018-1089-9.

[17] M. Huang, W. Liu, T. Wang, H. Song, X. Li, A. Liu, A queuing delay utilization scheme for on-path service aggregation in services-oriented computing networks, IEEE Access 7 (2019) 23816–23833.

[18] Y. Mesmoudi, M. Lamnaour, Y. El Khamlichi, A. Tahiri, A. Touhafi, A. Braeken, A middleware based on service oriented architecture for heterogeneity issues within the internet of things (MSOAH-IoT), J. King Saud Univ. Comput. Inf. Sci. 32 (10) (2020) 1108–1116, http://dx.doi.org/10.1016/j.jksuci.2018.11.011.

[19] S. Vergara, L. González, R. Ruggia, Towards formalizing microservices architectural patterns with event-b, in: 2020 IEEE International Conference on Software Architecture Companion (ICSA-C), IEEE Computer Society, 2020, pp. 71–74, http://dx.doi.org/10.1109/ICSA-C50368.2020.00022.

[20] H.A. Le, S. Nakajima, N.T. Truong, Formal analysis of imprecise system requirements with event-b, SpringerPlus 5 (1) (2016) 1–16, http://dx.doi.org/10.1186/s40064-016-2657-8.

[21] M. Camilli, C. Bellettini, L. Capra, M. Monga, A formal framework for specifying and verifying microservices based process flows, in: International Conference on Software Engineering and Formal Methods, Springer, 2017, pp. 187–202, http://dx.doi.org/10.1007/978-3-319-74781-1_14.

[22] R. Petrasch, Model-based engineering for microservice architectures using enterprise integration patterns for inter-service communication, in: 2017 14th International Joint Conference on Computer Science and Software Engineering (JCSSE), IEEE, 2017, pp. 1–4, http://dx.doi.org/10.1109/JCSSE.2017.8025912.

[23] M. Singh, A. Sharma, R. Saxena, Towards the formalization of road traffic management system for safety critical properties by Z notation, in: 2015 International Conference on Green Computing and Internet of Things (ICGCIoT), IEEE, 2015, pp. 1516–1521, http://dx.doi.org/10.1109/ICGCIoT.2015.7380707.

[24] C.K. Rath, A.K. Mandal, A. Sarkar, An event-b based device description model in IoT with the support of multimodal system, in: Advanced Computing and Systems for Security: Vol. 13, Springer, 2022, pp. 3–19, http://dx.doi.org/10.1007/978-981-16-4287-6_1.

[25] M. Diwan, M. D'Souza, A framework for modeling and verifying IoT communication protocols, in: International Symposium on Dependable Software Engineering: Theories, Tools, and Applications, Springer, 2017, pp. 266–280, http://dx.doi.org/10.1007/978-3-319-69483-2_16.

[26] I. Ait-Sadoune, Y. Ait-Ameur, Formal modelling and verification of transactional web service composition: A refinement and proof approach with event-b, in: Correct Software in Web Applications and Web Services, Springer, 2015, pp. 1–27, http://dx.doi.org/10.1007/978-3-319-17112-8_1.

[27] I. Tounsi, M. Hadj Kacem, A. Hadj Kacem, Building correct by construction SOA design patterns: Modeling and refinement, in: European Conference on Software Architecture, Springer, 2013, pp. 33–44, http://dx.doi.org/10.1007/978-3-642-39031-9_4.

[28] J.W. Bryans, W. Wei, Formal analysis of BPMN models using event-b, in: International Workshop on Formal Methods for Industrial Critical Systems, Springer, 2010, pp. 33–49, http://dx.doi.org/10.1007/978-3-642-15898-8_3.

[29] D. Corral-Plaza, I. Medina-Bulo, G. Ortiz, J. Boubeta-Puig, A stream processing architecture for heterogeneous data sources in the internet of things, Comput. Stand. Interfaces 70 (2020) 103426.

[30] A.J. Fernández-García, L. Iribarne, A. Corral, J. Criado, J.Z. Wang, A flexible data acquisition system for storing the interactions on mashup user interfaces, Comput. Stand. Interfaces 59 (2018) 10–34.

[31] I. Zyrianoff, F. Borelli, G. Biondi, A. Heideker, C. Kamienski, Scalability of real-time iot-based applications for smart cities, in: 2018 IEEE Symposium on Computers and Communications (ISCC), IEEE, 2018, pp. 00688–00693, http://dx.doi.org/10.1109/ISCC.2018.8538451.

[32] C.-L. Tseng, F.J. Lin, Extending scalability of iot/m2 m platforms with fog computing, in: 2018 IEEE 4th World Forum on Internet of Things (WF-IoT), IEEE, 2018, pp. 825–830, http://dx.doi.org/10.1109/WF-IoT.2018.8355143.

[33] I. Miladinovic, S. Schefer-Wenzl, A highly scalable iot architecture through network function virtualization, Open J. Internet of Things (OJIOT) 3 (1) (2017) 127–135.

[34] R. Filev Maia, C. Ballester Lurbe, A. Agrahari Baniya, J. Hornbuckle, Irrisens: An iot platform based on microservices applied in commercial-scale crops working in a multi-cloud environment, Sensors 20 (24) (2020) 7163, http://dx.doi.org/10.3390/s20247163.

[35] H. Chegini, A. Mahanti, A framework of automation on context-aware internet of things (IoT) systems, in: Proceedings of the 12th IEEE/ACM International Conference on Utility and Cloud Computing Companion, 2019, pp. 157–162, http://dx.doi.org/10.1145/3368235.3368848.

[36] G. Ortiz, M. Zouai, O. Kazar, A. Garcia-de Prado, J. Boubeta-Puig, Atmosphere: Context and situational-aware collaborative IoT architecture for edge-fog-cloud computing, Comput. Stand. Interfaces 79 (2022) 103550.

[37] N.C. Coulson, S. Sotiriadis, N. Bessis, Adaptive microservice scaling for elastic applications, IEEE Internet Things J. 7 (5) (2020) 4195–4202, http://dx.doi.org/10.1109/JIOT.2020.2964405.

[38] C. Sarkar, S.A.U. Nambi, R.V. Prasad, A. Rahim, A scalable distributed architecture towards unifying IoT applications, in: 2014 IEEE World Forum on Internet of Things (WF-IoT), IEEE, 2014, pp. 508–513, http://dx.doi.org/10.1109/WF-IoT.2014.6803220.

[39] T. Vresk, I. Čavrak, Architecture of an interoperable IoT platform based on microservices, in: 2016 39th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), IEEE, 2016, pp. 1196–1201, http://dx.doi.org/10.1109/mipro.2016.7522321.

[40] A. Benayache, A. Bilami, S. Barkat, P. Lorenz, H. Taleb, MsM: A microservice middleware for smart WSN-based IoT application, J. Netw. Comput. Appl. 144 (2019) 138–154, http://dx.doi.org/10.1016/j.jnca.2019.06.015.

[41] G. Ortiz, J. Boubeta-Puig, J. Criado, D. Corral-Plaza, A. Garcia-de Prado, I. Medina-Bulo, L. Iribarne, A microservice architecture for real-time IoT data processing: A reusable web of things approach for smart ports, Comput. Stand. Interfaces 81 (2022) 103604, http://dx.doi.org/10.1016/j.csi.2021.103604.

[42] N. Gavrilović, A. Mishra, Software architecture of the internet of things (IoT) for smart city, healthcare and agriculture: analysis and improvement directions, J. Ambient Intell. Humaniz. Comput. 12 (1) (2021) 1315–1336, http://dx.doi.org/10.1007/s12652-020-02197-3.

[43] N. Dipsis, K. Stathis, A restful middleware for AI controlled sensors, actuators and smart devices, J. Ambient Intell. Humaniz. Comput. 11 (7) (2020) 2963–2986, http://dx.doi.org/10.1007/s12652-019-01439-3.

[44] G. Porruvecchio, A. Romanino, C. Casari, R. Sanna, A microservice-based platform for IoT application development, in: 2021 IEEE 12th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), IEEE, 2021, pp. 0332–0336, http://dx.doi.org/10.1109/UEMCON53757.2021.9666727.

[45] R. Morabito, R. Petrolo, V. Loscri, N. Mitton, LEGIoT: A lightweight edge gateway for the internet of things, Future Gener. Comput. Syst. 81 (2018) 1–15, http://dx.doi.org/10.1016/j.future.2017.10.011.

[46] M. Tavana, V. Hajipour, S. Oveisi, IoT-based enterprise resource planning: Challenges, open issues, applications, architecture, and future research directions, Internet of Things 11 (2020) 100262, http://dx.doi.org/10.1016/j.iot.2020.100262.

[47] M.S. Aslanpour, A.N. Toosi, C. Cicconetti, B. Javadi, P. Sbarski, D. Taibi, M. Assuncao, S.S. Gill, R. Gaire, S. Dustdar, Serverless edge computing: vision and challenges, in: 2021 Australasian Computer Science Week Multiconference, 2021, pp. 1–10, http://dx.doi.org/10.1145/3437378.3444367.

[48] L. Pan, L. Wang, S. Chen, F. Liu, Retention-aware container caching for serverless edge computing, in: Proc. of IEEE INFOCOM, IEEE, 2022.

[49] D.C. Li, C.-T. Huang, C.-W. Tseng, L.-D. Chou, Fuzzy-based microservice resource management platform for edge computing in the internet of things, Sensors 21 (11) (2021) 3800, http://dx.doi.org/10.3390/s21113800.

[50] C.-C. Liu, C.-C. Huang, C.-W. Tseng, Y.-T. Yang, L.-D. Chou, Service resource management in edge computing based on microservices, in: 2019 IEEE International Conference on Smart Internet of Things (SmartIoT), IEEE, 2019, pp. 388–392, http://dx.doi.org/10.1109/SmartIoT.2019.00068.

[51] M.A. Jarwar, M.G. Kibria, S. Ali, I. Chong, Microservices in web objects enabled iot environment for enhancing reusability, Sensors 18 (2) (2018) 352, http://dx.doi.org/10.3390/s18020352.

[52] P. Smet, B. Dhoedt, P. Simoens, Docker layer placement for on-demand provisioning of services on edge clouds, IEEE Trans. Netw. Serv. Manag. 15 (3) (2018) 1161–1174.

[53] M. Fazio, A. Celesti, R. Ranjan, C. Liu, L. Chen, M. Villari, Open issues in scheduling microservices in the cloud, IEEE Cloud Comput. 3 (5) (2016) 81–88, http://dx.doi.org/10.1109/MCC.2016.112.

[54] C.K. Rath, Implementation of microservice based IoT architecture in rodin platform, 2022, Accessed: 2022-01-20. URL https://github.com/chouhanrath/DeviceProfile_Microservice_Event-BModel.

[55] C.K. Rath, Implementation of microservice based IoT architecture, 2022, Accessed: 2022-01-20. URL https://github.com/chouhanrath/DeviceProfile_Kafka_Microservices.

[56] P. Chapman, Microservices with spring, 2019, Accessed: 2021-10-23. URL https://spring.io/blog/2015/07/14/microservices-with-spring.

[57] J. Karee, How to parallelise kafka consumers, 2019, Accessed: 2021-10-8. URL https://medium.com/@jhansireddy007/how-to-parallelise-kafka-consumers-59c8b0bbc37a.